Re-practical Practice Questions

The Menu and Binary scenario: A_1 to B_3

COS 110 | Swift Tuition | 2026 edition

These practice questions are based on the published scenario for the COS 110 re-practical (Section A: `Item`, `Menu` and the exceptions; Section B: `Binary`, `Node<T>` and `BinaryStrings`). The published UML is partial, so the real exam's classes may have more members. Read the given headers on the day. Each question has a folder in the zip file with the starter files and a makefile, laid out like the practical. Write only the one file the question names, build with `make`, and check the output and valgrind.

Contents

- **A_1 Exceptions**
  1. A1-01. Exceptions: the constructors of the exception classes and `printException` (100)
  2. A1-02. Exceptions: `Menu::findFirst` throws `InvalidType` or the right not-found exception (100)
  3. A1-03. Exceptions: `Menu::findLast` searches from the back and throws (100)
  4. A1-04. Exceptions: `Menu::averagePrice` throws `NothingMatchingFilter` (100)
  5. A1-05. Exceptions: `Menu::numberAbove` counts items and throws `NothingAboveMinimumValue` (100)
  6. A1-06. Exceptions: `Menu::numberBelow` counts items and throws `NothingBelowMaximumValue` (100)
  7. A1-07. Exceptions: `Menu::numberEqual` counts items and throws `NothingMatchingFilter` (100)
  8. A1-08. Exceptions: `Menu::printSummary` catches each exception type in the right order (100)
- **A_2 Operator overloading**
  1. A2-01. Operator overloading: `operator<<` for `Item` (100)
  2. A2-02. Operator overloading: `operator<<` for `Menu` (prints empty slots) (100)
  3. A2-03. Operator overloading: `operator<<` for `Exception` (100)
  4. A2-04. Operator overloading: `Menu::operator+(Item*)` adds an item or deletes it (100)
  5. A2-05. Operator overloading: `Menu::operator[]` returns `Item*&` (100)
  6. A2-06. Operator overloading: `Item::operator==` and `Item::operator<` (100)
  7. A2-07. Operator overloading: `Menu::operator+=` grows the array when the menu is full (100)
  8. A2-08. Operator overloading: `Menu::operator-(string)` removes an item by name (100)
- **A_3 Assignment operator**
  1. A3-01. Assignment operator: `Item::operator=` (100)
  2. A3-02. Assignment operator: `Menu::operator=` (100)
  3. A3-03. Copy constructor: `Menu::Menu(const Menu&)` (100)
  4. A3-04. Assignment operator: `Menu::operator=` that keeps its own size (100)
  5. A3-05. Assignment operator: `Binary::operator=` (a `bool*` array) (100)
- **B_1 Recursion**
  1. B1-01. Recursion: `Binary::toDecimal` (100)
  2. B1-02. Recursion: `Binary::countOnes` (100)
  3. B1-03. Recursion: `Binary::isPalindrome` (100)
  4. B1-04. Recursion: `BinaryStrings::countNegative` (100)
  5. B1-05. Recursion: `BinaryStrings::calculatePos` with a node and a bit index (100)
  6. B1-06. Recursion: `BinaryStrings::printReverse` (100)
  7. B1-07. Recursion: `Binary::setBits` (100)

- **B_2 Linked lists**
    1. B2-01. Linked lists: `BinaryStrings::append` (100)
    2. B2-02. Linked lists: `BinaryStrings::insertAt` (100)
    3. B2-03. Linked lists: `BinaryStrings::removeAt` (100)
    4. B2-04. Linked lists: `BinaryStrings::remove` the first equal string (100)
    5. B2-05. Linked lists: `BinaryStrings::reverse` (100)
    6. B2-06. Linked lists: `BinaryStrings::sort` by decimal value (100)
    7. B2-07. Linked lists: `BinaryStrings::calculatePos` (100)
    8. B2-08. Linked lists: `BinaryStrings` copy constructor (100)
- **B_3 Template specialisation**
    1. B3-01. Template specialisation: `printValue<Binary>` (100)
    2. B3-02. Template specialisation: `largest<Binary>` (100)
    3. B3-03. Template specialisation: `isEqual<Binary>` (100)
    4. B3-04. Template specialisation: `addition<Binary, Binary>` throws (100)
    5. B3-05. Template specialisation: member `Node<Binary>::toInt` (100)
    6. B3-06. Template specialisation: `countTrue<Binary>` in a file that a header includes (100)

A_1 Exceptions

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A1-01. Exceptions: the constructors of the exception classes and `printException` (100)

For this question, implement the exception hierarchy in the file `A_1.cpp`: the constructor `Exception(string)`, the member function `printException()`, and the constructor of each of the eight subclasses. The destructor `~Exception()` and the stream insertion `operator<<` are given in `Exception.cpp`.

UML (only the classes this question needs)

```
Exception
- msg: string
+ Exception(string)   <- implement
+ ~Exception()   {virtual}
+ printException(): void   <- implement
+ operator<<(ostream&, const Exception&): ostream&   {friend}

InvalidType : Exception
+ InvalidType(string)   <- implement

NothingMatchingFilter : Exception
+ NothingMatchingFilter(string)   <- implement

NothingAboveMinimumValue : NothingMatchingFilter
+ NothingAboveMinimumValue()   <- implement

NothingBelowMaximumValue : NothingMatchingFilter
+ NothingBelowMaximumValue()   <- implement

NotFoundException : Exception
+ NotFoundException()   <- implement

StarterNotFound : NotFoundException
+ StarterNotFound()   <- implement

MainNotFound : NotFoundException
+ MainNotFound()   <- implement

DessertNotFound : NotFoundException
+ DessertNotFound()   <- implement
```

Differences from the partial UML in the exam paper:

- `~Exception()` is virtual, so that the classes are polymorphic.
- `operator<<` is not drawn, but the exam paper says every class has one.
- The exam paper does not give the messages. The messages below are the ones this bank uses. In the exam, the header files and the question text decide.

Given code you will use

- `Exception::~Exception()` is given. It is `virtual` and does nothing.
- `operator<<(ostream& out, const Exception& e)` is given. It prints `Exception: <msg>` with no newline.

What the functions must do

1. `Exception(string msg)` stores `msg` in the member `msg`.
2. `printException()` prints `Exception:` and then `msg`, and ends the line with `endl`. It prints to `cout`.
3. Each subclass constructor stores this message:

Class	Message stored in `msg`
`InvalidType(t)`	`Invalid type:` followed by the value of `t`, for example `Invalid type: Drink`
`NothingMatchingFilter(f)`	`Nothing matching filter:` followed by the value of `f`, for example `Nothing matching filter: Dessert`
`NothingAboveMinimumValue()`	`Nothing matching filter: above minimum value`
`NothingBelowMaximumValue()`	`Nothing matching filter: below maximum value`
`NotFoundException()`, `StarterNotFound()`, `MainNotFound()`, `DessertNotFound()`	`Item not found`

More rules:

- `msg` is `private` in `Exception`. A subclass cannot assign to `msg`. It passes its message to the constructor of its direct base class, in the member initialiser list, for example `: Exception("...")`.
- `NothingAboveMinimumValue` and `NothingBelowMaximumValue` are derived from `NothingMatchingFilter`, not directly from `Exception`. Their constructors must call the `NothingMatchingFilter` constructor. The `NothingMatchingFilter` constructor adds `Nothing matching filter:` in front of the text it gets.
- The three not-found classes are derived from `NotFoundException`. Their message comes from the `NotFoundException` constructor.
- `Exception` and `NothingMatchingFilter` have no default constructor. So the constructors of `InvalidType`, `NothingMatchingFilter`, `NotFoundException`, `NothingAboveMinimumValue` and `NothingBelowMaximumValue` must call their direct base constructor in the initialiser list. If they do not, the code does not compile.
- `NotFoundException()` takes no argument. For the three not-found classes, the call `: NotFoundException()` is optional. If it is left out, C++ calls `NotFoundException()` anyway.

The given `operator<<` functions and the `ExpectedOutput` block apply to this question only. Another question in this bank may print the same class in a different way.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`. `-Wall -Werror` turns every warning into an error.

3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A1-02. Exceptions: `Menu::findFirst` throws `InvalidType` or the right not-found exception (100)

For this question, implement the member function `findFirst` of the `Menu` class in the file `A_1.cpp`. It returns a copy of the first item of a type. It throws an exception when the type name is not valid or when the menu has no item of that type.

UML (only the classes this question needs)

```
Exception
- msg: string
+ Exception(string)
+ ~Exception()   {virtual}
+ printException(): void
+ operator<<(ostream&, const Exception&): ostream&   {friend}

InvalidType : Exception
+ InvalidType(string)

NothingMatchingFilter : Exception
+ NothingMatchingFilter(string)

NothingAboveMinimumValue : NothingMatchingFilter
+ NothingAboveMinimumValue()

NothingBelowMaximumValue : NothingMatchingFilter
+ NothingBelowMaximumValue()

NotFoundException : Exception
+ NotFoundException()

StarterNotFound : NotFoundException
+ StarterNotFound()

MainNotFound : NotFoundException
+ MainNotFound()

DessertNotFound : NotFoundException
+ DessertNotFound()

Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ getType(): char   {const}
+ getName(): string   {const}
+ getPrice(): float   {const}
+ operator<<(ostream&, const Item&): ostream&   {friend}

Menu
- maxSize: int
- items: Item**
```

```
+ Menu(int)
+ ~Menu()
+ operator+(Item*): Menu&
+ findFirst(string): Item   <- implement
+ operator<<(ostream&, const Menu&): ostream&   {friend}
```

Differences from the partial UML in the exam paper:

- `operator<<` is not drawn, but the exam paper says every class has one.
- `~Exception()` is virtual, so that the classes are polymorphic.
- `operator+` returns `Menu&`. The exam paper shows no return type.
- `getPrice()` returns `float` here. The exam paper shows `string`, which is probably a mistake, because `price` is a `float`.
- The exam paper's `Menu` also has `findLast`, `averagePrice`, `numberAbove`, `numberBelow`, `numberEqual`. This question does not use them.

Given code you will use

- `Item`'s `operator<<` prints `<name> (<type>) R<price>` with no newline, for example `Garlic bread (S) R45.5`. The type is `'S'` (starter), `'M'` (main) or `'D'` (dessert).
- `Menu(int maxSize)` makes an array of `maxSize` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.
- `operator+(Item*)` puts the item in the first `NULL` slot and returns the menu, so `menu + a + b;` works. If the menu is full, it deletes the item. The menu owns its items and deletes them in `~Menu()`. Nothing in this question removes items, so the filled slots are always at the front.
- `Menu`'s `operator<<` prints `Menu (max <maxSize>):` and then one line per slot: two spaces, then `[`, the index, `]` and one space, then the item, or `empty` for a `NULL` slot.
- The exception classes are given. `printException()` prints `Exception: <msg>` and a newline. The messages are:

Class	Message stored in `msg`
`InvalidType(t)`	`Invalid type: t`
`NothingMatchingFilter(f)`	`Nothing matching filter: f`
`NothingAboveMinimumValue()`	`Nothing matching filter: above minimum value`
`NothingBelowMaximumValue()`	`Nothing matching filter: below maximum value`
`NotFoundException()`, `StarterNotFound()`, `MainNotFound()`, `DessertNotFound()`	`Item not found`

In the table, `t` and `f` stand for the text that the constructor receives. The three not-found classes have the same message. Only the `catch` that catches them tells them apart.

What `findFirst(string type)` must do

The `string` parameter is a type name. `"Starter"` means `'S'`, `"Main"` means `'M'` and `"Dessert"` means `'D'`. The match is exact and case-sensitive, so `"main"` and `"Drink"` are not valid type names.

1. If `type` is not one of the three valid names, throw `InvalidType(type)`.

2. Otherwise, go through the slots from index 0 upward. Return a copy of the first item whose type matches.
3. If no item matches, throw the not-found class for that type: `StarterNotFound()` for `"Starter"`, `MainNotFound()` for `"Main"`, `DessertNotFound()` for `"Dessert"`.

More rules:

- Check the type name first, before the loop. An invalid name throws `InvalidType` even for an empty menu.
- Return the item itself by value: `return *items[i];`. The copy constructor makes the copy.
- Skip `NULL` slots. A call through a `NULL` pointer stops the program.
- Throw the exception object by value, for example `throw MainNotFound();`. Do not use `new`. The caller catches by reference.
- Do not catch the exception and do not print anything in `findFirst`.

The given `operator<<` functions and the `ExpectedOutput` block apply to this question only. Another question in this bank may print the same class in a different way.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`. `-Wall -Werror` turns every warning into an error.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A1-03. Exceptions: `Menu::findLast` searches from the back and throws (100)

For this question, implement the member function `findLast` of the `Menu` class in the file `A_1.cpp`. It returns a copy of the last item of a type, that is the matching item with the highest index. It throws an exception when the type name is not valid or when the menu has no item of that type.

UML (only the classes this question needs)

```
Exception
- msg: string
+ Exception(string)
+ ~Exception()   {virtual}
+ printException(): void
+ operator<<(ostream&, const Exception&): ostream&   {friend}

InvalidType : Exception
+ InvalidType(string)

NothingMatchingFilter : Exception
+ NothingMatchingFilter(string)

NothingAboveMinimumValue : NothingMatchingFilter
+ NothingAboveMinimumValue()

NothingBelowMaximumValue : NothingMatchingFilter
+ NothingBelowMaximumValue()

NotFoundException : Exception
+ NotFoundException()

StarterNotFound : NotFoundException
+ StarterNotFound()

MainNotFound : NotFoundException
+ MainNotFound()

DessertNotFound : NotFoundException
+ DessertNotFound()

Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ getType(): char   {const}
+ getName(): string   {const}
+ getPrice(): float   {const}
+ operator<<(ostream&, const Item&): ostream&   {friend}

Menu
- maxSize: int
- items: Item**
```

```
+ Menu(int)
+ ~Menu()
+ operator+(Item*): Menu&
+ findLast(string): Item   <- implement
+ operator<<(ostream&, const Menu&): ostream&   {friend}
```

Differences from the partial UML in the exam paper:

- `operator<<` is not drawn, but the exam paper says every class has one.
- `~Exception()` is virtual, so that the classes are polymorphic.
- `operator+` returns `Menu&`. The exam paper shows no return type.
- `getPrice()` returns `float` here. The exam paper shows `string`, which is probably a mistake, because `price` is a `float`.
- The exam paper's `Menu` also has `findFirst`, `averagePrice`, `numberAbove`, `numberBelow`, `numberEqual`. This question does not use them.

Given code you will use

- `Item`'s `operator<<` prints `<name> (<type>) R<price>` with no newline, for example `Garlic bread (S) R45.5`. The type is `'S'` (starter), `'M'` (main) or `'D'` (dessert).
- `Menu(int maxSize)` makes an array of `maxSize` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.
- `operator+(Item*)` puts the item in the first `NULL` slot and returns the menu, so `menu + a + b;` works. If the menu is full, it deletes the item. The menu owns its items and deletes them in `~Menu()`. Nothing in this question removes items, so the filled slots are always at the front.
- `Menu`'s `operator<<` prints `Menu (max <maxSize>):` and then one line per slot: two spaces, then `[`, the index, `]` and one space, then the item, or `empty` for a `NULL` slot.
- The exception classes are given. `printException()` prints `Exception: <msg>` and a newline. The messages are:

Class	Message stored in `msg`
`InvalidType(t)`	`Invalid type: t`
`NothingMatchingFilter(f)`	`Nothing matching filter: f`
`NothingAboveMinimumValue()`	`Nothing matching filter: above minimum value`
`NothingBelowMaximumValue()`	`Nothing matching filter: below maximum value`
`NotFoundException()`, `StarterNotFound()`, `MainNotFound()`, `DessertNotFound()`	`Item not found`

In the table, `t` and `f` stand for the text that the constructor receives. The three not-found classes have the same message. Only the `catch` that catches them tells them apart.

What `findLast(string type)` must do

The `string` parameter is a type name. `"Starter"` means `'S'`, `"Main"` means `'M'` and `"Dessert"` means `'D'`. The match is exact and case-sensitive, so `"main"` and `"Drink"` are not valid type names.

1. If `type` is not one of the three valid names, throw `InvalidType(type)`.
2. Otherwise, return a copy of the matching item with the highest index.

3. If no item matches, throw `StarterNotFound()`, `MainNotFound()` or `DessertNotFound()`, for `"Starter"`, `"Main"` or `"Dessert"`.

More rules:

- Check the type name first, before the loop. An invalid name throws `InvalidType` even for an empty menu.
- A loop that starts at the back meets the empty (`NULL`) slots first. Skip them.
- The valid indexes are `0` to `maxSize - 1`. Do not read `items[maxSize]`.
- Throw the exception object by value. Do not catch it and do not print anything in `findLast`.

The given `operator<<` functions and the `ExpectedOutput` block apply to this question only. Another question in this bank may print the same class in a different way.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`. `-Wall -Werror` turns every warning into an error.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A1-04. Exceptions: `Menu::averagePrice` throws `NothingMatchingFilter` (100)

For this question, implement the member function `averagePrice` of the `Menu` class in the file `A_1.cpp`. It returns the average price of the items of one type. It throws an exception when the type name is not valid or when the menu has no item of that type.

UML (only the classes this question needs)

```
Exception
- msg: string
+ Exception(string)
+ ~Exception()   {virtual}
+ printException(): void
+ operator<<(ostream&, const Exception&): ostream&   {friend}

InvalidType : Exception
+ InvalidType(string)

NothingMatchingFilter : Exception
+ NothingMatchingFilter(string)

NothingAboveMinimumValue : NothingMatchingFilter
+ NothingAboveMinimumValue()

NothingBelowMaximumValue : NothingMatchingFilter
+ NothingBelowMaximumValue()

NotFoundException : Exception
+ NotFoundException()

StarterNotFound : NotFoundException
+ StarterNotFound()

MainNotFound : NotFoundException
+ MainNotFound()

DessertNotFound : NotFoundException
+ DessertNotFound()

Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ getType(): char   {const}
+ getName(): string   {const}
+ getPrice(): float   {const}
+ operator<<(ostream&, const Item&): ostream&   {friend}

Menu
- maxSize: int
- items: Item**
```

```
+ Menu(int)
+ ~Menu()
+ operator+(Item*): Menu&
+ averagePrice(string): float   <- implement
+ operator<<(ostream&, const Menu&): ostream&   {friend}
```

Differences from the partial UML in the exam paper:

- `operator<<` is not drawn, but the exam paper says every class has one.
- `~Exception()` is virtual, so that the classes are polymorphic.
- `operator+` returns `Menu&`. The exam paper shows no return type.
- `getPrice()` returns `float` here. The exam paper shows `string`, which is probably a mistake, because `price` is a `float`.
- The exam paper's `Menu` also has `findFirst`, `findLast`, `numberAbove`, `numberBelow`, `numberEqual`. This question does not use them.

Given code you will use

- `Item`'s `operator<<` prints `<name> (<type>) R<price>` with no newline, for example `Garlic bread (S) R45.5`. The type is `'S'` (starter), `'M'` (main) or `'D'` (dessert).
- `Menu(int maxSize)` makes an array of `maxSize` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.
- `operator+(Item*)` puts the item in the first `NULL` slot and returns the menu, so `menu + a + b;` works. If the menu is full, it deletes the item. The menu owns its items and deletes them in `~Menu()`. Nothing in this question removes items, so the filled slots are always at the front.
- `Menu`'s `operator<<` prints `Menu (max <maxSize>):` and then one line per slot: two spaces, then `[`, the index, `]` and one space, then the item, or `empty` for a `NULL` slot.
- The exception classes are given. `printException()` prints `Exception: <msg>` and a newline. The messages are:

Class	Message stored in `msg`
`InvalidType(t)`	`Invalid type: t`
`NothingMatchingFilter(f)`	`Nothing matching filter: f`
`NothingAboveMinimumValue()`	`Nothing matching filter: above minimum value`
`NothingBelowMaximumValue()`	`Nothing matching filter: below maximum value`
`NotFoundException()`, `StarterNotFound()`, `MainNotFound()`, `DessertNotFound()`	`Item not found`

In the table, `t` and `f` stand for the text that the constructor receives. The three not-found classes have the same message. Only the `catch` that catches them tells them apart.

What `averagePrice(string type)` must do

The `string` parameter is a type name. `"Starter"` means `'S'`, `"Main"` means `'M'` and `"Dessert"` means `'D'`. The match is exact and case-sensitive, so `"main"` and `"Drink"` are not valid type names.

1. If `type` is not one of the three valid names, throw `InvalidType(type)`.
2. Otherwise, add up the prices of the items of that type and count them.

3. If the count is 0, throw `NothingMatchingFilter(type)`. The message then names the type, for example `Nothing matching filter: Dessert`.
4. Otherwise, return the total divided by the count. Keep the total in a `float`, so that the decimals of the prices are kept.

More rules:

- Check the type name first, before the loop. An invalid name throws `InvalidType` even for an empty menu.
- Skip `NULL` slots.
- An item with the price 0 still counts as an item.
- Throw the exception object by value. Do not catch it and do not print anything in `averagePrice`.

The given `operator<<` functions and the `ExpectedOutput` block apply to this question only. Another question in this bank may print the same class in a different way.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`. `-Wall -Werror` turns every warning into an error.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A1-05. Exceptions: `Menu::numberAbove` counts items and throws `NothingAboveMinimumValue` (100)

For this question, implement the member function `numberAbove` of the `Menu` class in the file `A_1.cpp`. It counts the items of one type whose price is strictly greater than a value. It throws an exception when the type name is not valid or when the count is 0.

UML (only the classes this question needs)

```
Exception
- msg: string
+ Exception(string)
+ ~Exception()   {virtual}
+ printException(): void
+ operator<<(ostream&, const Exception&): ostream&   {friend}

InvalidType : Exception
+ InvalidType(string)

NothingMatchingFilter : Exception
+ NothingMatchingFilter(string)

NothingAboveMinimumValue : NothingMatchingFilter
+ NothingAboveMinimumValue()

NothingBelowMaximumValue : NothingMatchingFilter
+ NothingBelowMaximumValue()

NotFoundException : Exception
+ NotFoundException()

StarterNotFound : NotFoundException
+ StarterNotFound()

MainNotFound : NotFoundException
+ MainNotFound()

DessertNotFound : NotFoundException
+ DessertNotFound()

Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ getType(): char   {const}
+ getName(): string   {const}
+ getPrice(): float   {const}
+ operator<<(ostream&, const Item&): ostream&   {friend}

Menu
- maxSize: int
- items: Item**
```

```
+ Menu(int)
+ ~Menu()
+ operator+(Item*): Menu&
+ numberAbove(string, float): int   <- implement
+ operator<<(ostream&, const Menu&): ostream&   {friend}
```

Differences from the partial UML in the exam paper:

- `operator<<` is not drawn, but the exam paper says every class has one.
- `~Exception()` is virtual, so that the classes are polymorphic.
- `operator+` returns `Menu&`. The exam paper shows no return type.
- `getPrice()` returns `float` here. The exam paper shows `string`, which is probably a mistake, because `price` is a `float`.
- The exam paper's `Menu` also has `findFirst`, `findLast`, `averagePrice`, `numberBelow`, `numberEqual`. This question does not use them.

Given code you will use

- `Item`'s `operator<<` prints `<name> (<type>) R<price>` with no newline, for example `Garlic bread (S) R45.5`. The type is `'S'` (starter), `'M'` (main) or `'D'` (dessert).
- `Menu(int maxSize)` makes an array of `maxSize` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.
- `operator+(Item*)` puts the item in the first `NULL` slot and returns the menu, so `menu + a + b;` works. If the menu is full, it deletes the item. The menu owns its items and deletes them in `~Menu()`. Nothing in this question removes items, so the filled slots are always at the front.
- `Menu`'s `operator<<` prints `Menu (max <maxSize>):` and then one line per slot: two spaces, then `[`, the index, `]` and one space, then the item, or `empty` for a `NULL` slot.
- The exception classes are given. `printException()` prints `Exception: <msg>` and a newline. The messages are:

Class	Message stored in `msg`
`InvalidType(t)`	`Invalid type: t`
`NothingMatchingFilter(f)`	`Nothing matching filter: f`
`NothingAboveMinimumValue()`	`Nothing matching filter: above minimum value`
`NothingBelowMaximumValue()`	`Nothing matching filter: below maximum value`
`NotFoundException()`, `StarterNotFound()`, `MainNotFound()`, `DessertNotFound()`	`Item not found`

In the table, `t` and `f` stand for the text that the constructor receives. The three not-found classes have the same message. Only the `catch` that catches them tells them apart.

What `numberAbove(string type, float value)` must do

The `string` parameter is a type name. `"Starter"` means `'S'`, `"Main"` means `'M'` and `"Dessert"` means `'D'`. The match is exact and case-sensitive, so `"main"` and `"Drink"` are not valid type names.

1. If `type` is not one of the three valid names, throw `InvalidType(type)`.

2. Otherwise, count the items of that type whose price is strictly greater than `value` (the test is `price > value`).
3. If the count is 0, throw `NothingAboveMinimumValue()`.
4. Otherwise, return the count.

More rules:

- Check the type name first, before the loop. An invalid name throws `InvalidType` even for an empty menu.
- Skip `NULL` slots.
- Count only items of the given type. An item of another type does not count, even when its price is strictly greater than `value`.
- Throw the exception object by value. Do not catch it and do not print anything in `numberAbove`.

The given `operator<<` functions and the `ExpectedOutput` block apply to this question only. Another question in this bank may print the same class in a different way.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`. `-Wall -Werror` turns every warning into an error.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A1-06. Exceptions: `Menu::numberBelow` counts items and throws `NothingBelowMaximumValue` (100)

For this question, implement the member function `numberBelow` of the `Menu` class in the file `A_1.cpp`. It counts the items of one type whose price is strictly less than a value. It throws an exception when the type name is not valid or when the count is 0.

UML (only the classes this question needs)

```
Exception
- msg: string
+ Exception(string)
+ ~Exception()   {virtual}
+ printException(): void
+ operator<<(ostream&, const Exception&): ostream&   {friend}

InvalidType : Exception
+ InvalidType(string)

NothingMatchingFilter : Exception
+ NothingMatchingFilter(string)

NothingAboveMinimumValue : NothingMatchingFilter
+ NothingAboveMinimumValue()

NothingBelowMaximumValue : NothingMatchingFilter
+ NothingBelowMaximumValue()

NotFoundException : Exception
+ NotFoundException()

StarterNotFound : NotFoundException
+ StarterNotFound()

MainNotFound : NotFoundException
+ MainNotFound()

DessertNotFound : NotFoundException
+ DessertNotFound()

Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ getType(): char   {const}
+ getName(): string   {const}
+ getPrice(): float   {const}
+ operator<<(ostream&, const Item&): ostream&   {friend}

Menu
- maxSize: int
- items: Item**
```

```
+ Menu(int)
+ ~Menu()
+ operator+(Item*): Menu&
+ numberBelow(string, float): int   <- implement
+ operator<<(ostream&, const Menu&): ostream&   {friend}
```

Differences from the partial UML in the exam paper:

- `operator<<` is not drawn, but the exam paper says every class has one.
- `~Exception()` is virtual, so that the classes are polymorphic.
- `operator+` returns `Menu&`. The exam paper shows no return type.
- `getPrice()` returns `float` here. The exam paper shows `string`, which is probably a mistake, because `price` is a `float`.
- The exam paper's `Menu` also has `findFirst`, `findLast`, `averagePrice`, `numberAbove`, `numberEqual`. This question does not use them.

Given code you will use

- `Item`'s `operator<<` prints `<name> (<type>) R<price>` with no newline, for example `Garlic bread (S) R45.5`. The type is `'S'` (starter), `'M'` (main) or `'D'` (dessert).
- `Menu(int maxSize)` makes an array of `maxSize` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.
- `operator+(Item*)` puts the item in the first `NULL` slot and returns the menu, so `menu + a + b;` works. If the menu is full, it deletes the item. The menu owns its items and deletes them in `~Menu()`. Nothing in this question removes items, so the filled slots are always at the front.
- `Menu`'s `operator<<` prints `Menu (max <maxSize>):` and then one line per slot: two spaces, then `[`, the index, `]` and one space, then the item, or `empty` for a `NULL` slot.
- The exception classes are given. `printException()` prints `Exception: <msg>` and a newline. The messages are:

Class	Message stored in `msg`
`InvalidType(t)`	`Invalid type: t`
`NothingMatchingFilter(f)`	`Nothing matching filter: f`
`NothingAboveMinimumValue()`	`Nothing matching filter: above minimum value`
`NothingBelowMaximumValue()`	`Nothing matching filter: below maximum value`
`NotFoundException()`, `StarterNotFound()`, `MainNotFound()`, `DessertNotFound()`	`Item not found`

In the table, `t` and `f` stand for the text that the constructor receives. The three not-found classes have the same message. Only the `catch` that catches them tells them apart.

What `numberBelow(string type, float value)` must do

The `string` parameter is a type name. `"Starter"` means `'S'`, `"Main"` means `'M'` and `"Dessert"` means `'D'`. The match is exact and case-sensitive, so `"main"` and `"Drink"` are not valid type names.

1. If `type` is not one of the three valid names, throw `InvalidType(type)`.

2. Otherwise, count the items of that type whose price is strictly less than `value` (the test is `price < value`).
3. If the count is 0, throw `NothingBelowMaximumValue()`.
4. Otherwise, return the count.

More rules:

- Check the type name first, before the loop. An invalid name throws `InvalidType` even for an empty menu.
- Skip `NULL` slots.
- Count only items of the given type. An item of another type does not count, even when its price is strictly less than `value`.
- Throw the exception object by value. Do not catch it and do not print anything in `numberBelow`.

The given `operator<<` functions and the `ExpectedOutput` block apply to this question only. Another question in this bank may print the same class in a different way.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`. `-Wall -Werror` turns every warning into an error.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A1-07. Exceptions: `Menu::numberEqual` counts items and throws `NothingMatchingFilter` (100)

For this question, implement the member function `numberEqual` of the `Menu` class in the file `A_1.cpp`. It counts the items of one type whose price is exactly equal to a value. It throws an exception when the type name is not valid or when the count is 0.

UML (only the classes this question needs)

```
Exception
- msg: string
+ Exception(string)
+ ~Exception()   {virtual}
+ printException(): void
+ operator<<(ostream&, const Exception&): ostream&   {friend}

InvalidType : Exception
+ InvalidType(string)

NothingMatchingFilter : Exception
+ NothingMatchingFilter(string)

NothingAboveMinimumValue : NothingMatchingFilter
+ NothingAboveMinimumValue()

NothingBelowMaximumValue : NothingMatchingFilter
+ NothingBelowMaximumValue()

NotFoundException : Exception
+ NotFoundException()

StarterNotFound : NotFoundException
+ StarterNotFound()

MainNotFound : NotFoundException
+ MainNotFound()

DessertNotFound : NotFoundException
+ DessertNotFound()

Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ getType(): char   {const}
+ getName(): string   {const}
+ getPrice(): float   {const}
+ operator<<(ostream&, const Item&): ostream&   {friend}

Menu
- maxSize: int
- items: Item**
```

```
+ Menu(int)
+ ~Menu()
+ operator+(Item*): Menu&
+ numberEqual(string, float): int   <- implement
+ operator<<(ostream&, const Menu&): ostream&   {friend}
```

Differences from the partial UML in the exam paper:

- `operator<<` is not drawn, but the exam paper says every class has one.
- `~Exception()` is virtual, so that the classes are polymorphic.
- `operator+` returns `Menu&`. The exam paper shows no return type.
- `getPrice()` returns `float` here. The exam paper shows `string`, which is probably a mistake, because `price` is a `float`.
- The exam paper's `Menu` also has `findFirst`, `findLast`, `averagePrice`, `numberAbove`, `numberBelow`. This question does not use them.

Given code you will use

- `Item`'s `operator<<` prints `<name> (<type>) R<price>` with no newline, for example `Garlic bread (S) R45.5`. The type is `'S'` (starter), `'M'` (main) or `'D'` (dessert).
- `Menu(int maxSize)` makes an array of `maxSize` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.
- `operator+(Item*)` puts the item in the first `NULL` slot and returns the menu, so `menu + a + b;` works. If the menu is full, it deletes the item. The menu owns its items and deletes them in `~Menu()`. Nothing in this question removes items, so the filled slots are always at the front.
- `Menu`'s `operator<<` prints `Menu (max <maxSize>):` and then one line per slot: two spaces, then `[`, the index, `]` and one space, then the item, or `empty` for a `NULL` slot.
- The exception classes are given. `printException()` prints `Exception: <msg>` and a newline. The messages are:

Class	Message stored in `msg`
`InvalidType(t)`	`Invalid type: t`
`NothingMatchingFilter(f)`	`Nothing matching filter: f`
`NothingAboveMinimumValue()`	`Nothing matching filter: above minimum value`
`NothingBelowMaximumValue()`	`Nothing matching filter: below maximum value`
`NotFoundException()`, `StarterNotFound()`, `MainNotFound()`, `DessertNotFound()`	`Item not found`

In the table, `t` and `f` stand for the text that the constructor receives. The three not-found classes have the same message. Only the `catch` that catches them tells them apart.

What `numberEqual(string type, float value)` must do

The `string` parameter is a type name. `"Starter"` means `'S'`, `"Main"` means `'M'` and `"Dessert"` means `'D'`. The match is exact and case-sensitive, so `"main"` and `"Drink"` are not valid type names.

1. If `type` is not one of the three valid names, throw `InvalidType(type)`.

2. Otherwise, count the items of that type whose price is exactly equal to `value` (the test is `price == value`).
3. If the count is 0, throw `NothingMatchingFilter(type)`.
4. Otherwise, return the count.

More rules:

- Check the type name first, before the loop. An invalid name throws `InvalidType` even for an empty menu.
- Skip `NULL` slots.
- Count only items of the given type. An item of another type does not count, even when its price is exactly equal to `value`.
- Throw the exception object by value. Do not catch it and do not print anything in `numberEqual`.
- Throw `NothingMatchingFilter(type)`, not one of its two subclasses. Its message names the type, for example `Nothing matching filter: Main`.
- The prices in this question have an exact `float` value (for example `95.5`), so `==` on two prices is safe here.

The given `operator<<` functions and the `ExpectedOutput` block apply to this question only. Another question in this bank may print the same class in a different way.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`. `-Wall -Werror` turns every warning into an error.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A1-08. Exceptions: `Menu::printSummary` catches each exception type in the right order (100)

For this question, implement the member function `printSummary` of the `Menu` class in the file `A_1.cpp`. It calls two given functions that can throw, prints one summary line, and catches every exception. Different exception types must print different lines.

UML (only the classes this question needs)

```
Exception
- msg: string
+ Exception(string)
+ ~Exception()   {virtual}
+ printException(): void
+ operator<<(ostream&, const Exception&): ostream&   {friend}

InvalidType : Exception
+ InvalidType(string)

NothingMatchingFilter : Exception
+ NothingMatchingFilter(string)

NothingAboveMinimumValue : NothingMatchingFilter
+ NothingAboveMinimumValue()

NothingBelowMaximumValue : NothingMatchingFilter
+ NothingBelowMaximumValue()

NotFoundException : Exception
+ NotFoundException()

StarterNotFound : NotFoundException
+ StarterNotFound()

MainNotFound : NotFoundException
+ MainNotFound()

DessertNotFound : NotFoundException
+ DessertNotFound()

Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ getType(): char   {const}
+ getName(): string   {const}
+ getPrice(): float   {const}
+ operator<<(ostream&, const Item&): ostream&   {friend}

Menu
- maxSize: int
- items: Item**
```

```
+ Menu(int)
+ ~Menu()
+ operator+(Item*): Menu&
+ findFirst(string): Item
+ numberAbove(string, float): int
+ printSummary(string, float): bool   <- implement
+ operator<<(ostream&, const Menu&): ostream&   {friend}
```

Differences from the partial UML in the exam paper:

- `Menu::printSummary` (the target) is not in the exam paper. The exam's catching code needs the same skill: `catch` blocks by reference, in the right order.
- `operator<<` is not drawn, but the exam paper says every class has one.
- `~Exception()` is virtual, so that the classes are polymorphic.
- `operator+` returns `Menu&`. The exam paper shows no return type.
- `getPrice()` returns `float` here. The exam paper shows `string`, which is probably a mistake, because `price` is a `float`.
- The exam paper's `Menu` also has `findLast`, `averagePrice`, `numberBelow`, `numberEqual`. This question does not use them.

Given code you will use

- `Item`'s `operator<<` prints `<name> (<type>) R<price>` with no newline, for example `Garlic bread (S) R45.5`. The type is `'S'` (starter), `'M'` (main) or `'D'` (dessert).
- `Menu(int maxSize)` makes an array of `maxSize` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.
- `operator+(Item*)` puts the item in the first `NULL` slot and returns the menu, so `menu + a + b;` works. If the menu is full, it deletes the item. The menu owns its items and deletes them in `~Menu()`. Nothing in this question removes items, so the filled slots are always at the front.
- `Menu`'s `operator<<` prints `Menu (max <maxSize>):` and then one line per slot: two spaces, then `[`, the index, `]` and one space, then the item, or `empty` for a `NULL` slot.
- The exception classes are given. `printException()` prints `Exception: <msg>` and a newline. The messages are:

Class	Message stored in `msg`
`InvalidType(t)`	`Invalid type: t`
`NothingMatchingFilter(f)`	`Nothing matching filter: f`
`NothingAboveMinimumValue()`	`Nothing matching filter: above minimum value`
`NothingBelowMaximumValue()`	`Nothing matching filter: below maximum value`
`NotFoundException()`, `StarterNotFound()`, `MainNotFound()`, `DessertNotFound()`	`Item not found`

In the table, `t` and `f` stand for the text that the constructor receives. The three not-found classes have the same message. Only the `catch` that catches them tells them apart. - `findFirst(string type)` returns a copy of the first item of that type. It throws `InvalidType(type)` for a type name that is not `"Starter"`, `"Main"` or `"Dessert"`. It throws `StarterNotFound()`, `MainNotFound()` or `DessertNotFound()` when there is no item of that type. - `numberAbove(string type, float value)` returns the number of items of

that type with a price greater than `value`. It throws `InvalidType(type)` for a bad type name and `NothingAboveMinimumValue()` when the count is 0.

What `printSummary(string type, float minPrice)` must do

Inside a `try` block, call `findFirst(type)` and then `numberAbove(type, minPrice)`. Then print one line and return:

Case	What is printed	Return
no exception	`<type>: first is <item>, <count> above R<minPrice>` and a newline	`true`
`findFirst` throws `DessertNotFound`	print `No desserts yet.` (with a space at the end), then call `e.printException()`	`false`
`findFirst` throws `StarterNotFound` or `MainNotFound` (another `NotFoundException`)	print `Not on the menu.` (with a space at the end), then call `e.printException()`	`false`
`numberAbove` throws `NothingAboveMinimumValue`	print `Nothing above R<minPrice>.` (with a space at the end), then call `e.printException()`	`false`
a call throws any other `Exception` (for example `InvalidType`)	print `Request failed.` (with a space at the end), then call `e.printException()`	`false`

`<item>` is the item printed with `Item`'s `operator<<`. `printException()` prints `Exception: <msg>` and the newline. Print `<minPrice>` with the default `float` output: `100` prints as `100` and `45.5` prints as `45.5`.

Example lines:

```
Main: first is Beef burger (M) R120, 1 above R100
No desserts yet. Exception: Item not found
```

More rules:

- No exception may leave `printSummary`. An uncaught exception stops the program abnormally, and the mark is 0.
- Catch each type by reference (with `&`). Here, catching by value gives a `-Wcatch-value` warning, and `-Werror` makes it an error. The warning appears because `Exception` has a virtual destructor in this question. If the exam's `Exception` has no virtual function, `catch (Exception e)` compiles without a warning. It still copies only the `Exception` part of the object. Always catch by reference.
- Print the summary line only when both calls worked. If `numberAbove` throws, nothing of the summary line may be printed.
- Order matters. `DessertNotFound` is also a `NotFoundException`, and every class is also an `Exception`. If a `catch` block for a base class comes before the block for its derived class, g++ gives a warning, and `-Werror` makes it an error.

The given `operator<<` functions and the `ExpectedOutput` block apply to this question only. Another question in this bank may print the same class in a different way.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`. `-Wall -Werror` turns every warning into an error.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Hints: `HINTS.md` in this folder.

A_2 Operator overloading

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A2-01. Operator overloading: `operator<<` for `Item` (100)

For this question, implement the stream insertion operator `operator<<` for the `Item` class in the file `A_2.cpp`. It is a `friend` function of `Item`. It prints one item on the stream it is given.

UML (only the classes this question needs)

```
Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ getType(): char   {const}
+ getName(): string   {const}
+ getPrice(): float   {const}
+ operator<<(ostream&, const Item&): ostream&   {friend}   <- implement

Menu
- maxSize: int
- items: Item**
+ Menu(int)
+ ~Menu()
+ operator+(Item*): Menu&
+ operator<<(ostream&, const Menu&): ostream&   {friend}
```

Differences from the partial UML in the exam paper:

- `operator<<` is not drawn, but the exam paper says every class has one.
- `operator+` returns `Menu&`. The exam paper shows no return type.
- `getPrice()` returns `float` here. The exam paper shows `string`, which is probably a mistake, because `price` is a `float`.
- The exam paper's `Menu` also has `findFirst`, `findLast`, `averagePrice`, `numberAbove`, `numberBelow`, `numberEqual`. This question does not use them.

Given code you will use

- `Menu(int maxSize)` makes an array of `maxSize` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.
- `operator+(Item*)` puts the item in the first `NULL` slot and returns the menu, so `menu + a + b;` works. If the menu is full, it deletes the item. The menu owns its items and deletes them in `~Menu()`. Nothing in this question removes items, so the filled slots are always at the front.
- `Menu`'s `operator<<` prints `Menu (max <maxSize>):` and then one line per slot: two spaces, then `[`, the index, `]` and one space, then the item, or `empty` for a `NULL` slot. It uses `Item`'s `operator<<` for each item.

What `operator<<(ostream& out, const Item& item)` must print

Print the name, a space, the type in round brackets, a space, the letter `R` and the price:

```
<name> (<type>) R<price>
```

For example `Garlic bread (S) R45.5` and `Beef burger (M) R120`.

More rules:

- Print the price with the default `float` output of the stream. Do not use `fixed` or `setprecision`. `120` prints as `120` and `45.5` prints as `45.5`.
- Print no newline. The caller adds `endl` when it wants one.
- Write to `out`, not to `cout`, and return `out`. The caller may pass another stream, for example an `ostringstream`. This is a stream that keeps the text in a `string` instead of showing it on the screen.
- Read `item.name`, `item.type` and `item.price` directly. A friend may read private members. This works whatever the header says about `const` getters. A getter that is not `const` cannot be called on the `const Item&`.
- Write the word `friend` only in the class. Do not write it again in `A_2.cpp`.

The given `operator<<` functions and the `ExpectedOutput` block apply to this question only. Another question in this bank may print the same class in a different way.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`. `-Wall -Werror` turns every warning into an error.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A2-02. Operator overloading: `operator<<` for `Menu` (prints empty slots) (100)

For this question, implement the stream insertion operator `operator<<` for the `Menu` class in the file `A_2.cpp`. It is a `friend` function of `Menu`. It prints a heading line and then one line for every slot of the array, also the empty ones.

UML (only the classes this question needs)

```
Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ getType(): char   {const}
+ getName(): string   {const}
+ getPrice(): float   {const}
+ operator<<(ostream&, const Item&): ostream&   {friend}

Menu
- maxSize: int
- items: Item**
+ Menu(int)
+ ~Menu()
+ operator+(Item*): Menu&
+ operator<<(ostream&, const Menu&): ostream&   {friend}   <- implement
```

Differences from the partial UML in the exam paper:

- `operator<<` is not drawn, but the exam paper says every class has one.
- `operator+` returns `Menu&`. The exam paper shows no return type.
- `getPrice()` returns `float` here. The exam paper shows `string`, which is probably a mistake, because `price` is a `float`.
- The exam paper's `Menu` also has `findFirst`, `findLast`, `averagePrice`, `numberAbove`, `numberBelow`, `numberEqual`. This question does not use them.

Given code you will use

- `Item`'s `operator<<` prints `<name> (<type>) R<price>` with no newline, for example `Garlic bread (S) R45.5`. The type is `'S'` (starter), `'M'` (main) or `'D'` (dessert).
- `Menu(int maxSize)` makes an array of `maxSize` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.
- `operator+(Item*)` puts the item in the first `NULL` slot and returns the menu, so `menu + a + b;` works. If the menu is full, it deletes the item. The menu owns its items and deletes them in `~Menu()`. Nothing in this question removes items, so the filled slots are always at the front.

What `operator<<(ostream& out, const Menu& menu)` must print

1. A heading line: `Menu (max <maxSize>):`.

2. Then one line for each slot, from index 0 to `maxSize - 1`: two spaces, then `[`, the index, `]` and one space, and then
3. the item, printed with `Item`'s `operator<<`, if the slot holds an item;
4. the word `empty`, if the slot is `NULL`.

Every line ends with `endl`, including the last one. Example for a menu of size 3 with two items:

```
Menu (max 3):
  [0] Garlic bread (S) R45.5
  [1] Beef burger (M) R120
  [2] empty
```

More rules:

- A menu of size 0 prints only `Menu (max 0):`.
- Do not dereference a `NULL` slot.
- Write to `out`, not to `cout`, and return `out`.
- `menu` is a `const` reference. A friend may read `menu.maxSize` and `menu.items`.
- Write the word `friend` only in the class. Do not write it again in `A_2.cpp`.

The given `operator<<` functions and the `ExpectedOutput` block apply to this question only. Another question in this bank may print the same class in a different way.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`. `-Wall -Werror` turns every warning into an error.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A2-03. Operator overloading: `operator<<` for `Exception` (100)

For this question, implement the stream insertion operator `operator<<` for the `Exception` class in the file `A_2.cpp`. It is a `friend` function of `Exception`. Because every exception class is derived from `Exception`, the same function prints every exception type.

UML (only the classes this question needs)

```
Exception
- msg: string
+ Exception(string)
+ ~Exception()   {virtual}
+ printException(): void
+ operator<<(ostream&, const Exception&): ostream&   {friend}   <- implement

InvalidType : Exception
+ InvalidType(string)

NothingMatchingFilter : Exception
+ NothingMatchingFilter(string)

NothingAboveMinimumValue : NothingMatchingFilter
+ NothingAboveMinimumValue()

NothingBelowMaximumValue : NothingMatchingFilter
+ NothingBelowMaximumValue()

NotFoundException : Exception
+ NotFoundException()

StarterNotFound : NotFoundException
+ StarterNotFound()

MainNotFound : NotFoundException
+ MainNotFound()

DessertNotFound : NotFoundException
+ DessertNotFound()
```

Differences from the partial UML in the exam paper:

- `~Exception()` is virtual, so that the classes are polymorphic.
- `operator<<` is not drawn, but the exam paper says every class has one. The one for `Exception` is the target.

Given code you will use

- The constructors and `printException()` are given. `printException()` prints `Exception: <msg>` and a newline. The messages are:

Class	Message stored in `msg`
`InvalidType(t)`	`Invalid type: t`

Class	Message stored in `msg`
`NothingMatchingFilter(f)`	`Nothing matching filter: f`
`NothingAboveMinimumValue()`	`Nothing matching filter: above minimum value`
`NothingBelowMaximumValue()`	`Nothing matching filter: below maximum value`
`NotFoundException()`, `StarterNotFound()`, `MainNotFound()`, `DessertNotFound()`	`Item not found`

In the table, `t` and `f` stand for the text that the constructor receives. The three not-found classes have the same message. Only the `catch` that catches them tells them apart.

What `operator<<(ostream& out, const Exception& e)` must print

Print `Exception:`, one space, and then the message:

```
Exception: <msg>
```

For example `Exception: Invalid type: Drink`. This is the text of `printException()`, without the newline.

More rules:

- Print no newline. The caller adds `endl` when it wants one.
- Write to `out`, not to `cout`, and return `out`.
- `e` is a `const` reference. `printException()` is not a `const` member function, so `e.printException()` does not compile here. Read `e.msg` directly; a friend may read a private member.
- Write the word `friend` only in the class. Do not write it again in `A_2.cpp`.

The given `operator<<` functions and the `ExpectedOutput` block apply to this question only. Another question in this bank may print the same class in a different way.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`. `-Wall -Werror` turns every warning into an error.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A2-04. Operator overloading: `Menu::operator+(Item*)` adds an item or deletes it (100)

For this question, implement the member operator `operator+` of the `Menu` class in the file `A_2.cpp`. It adds one item to the menu and returns the menu, so that `menu + a + b;` adds two items.

UML (only the classes this question needs)

```
Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ getType(): char   {const}
+ getName(): string   {const}
+ getPrice(): float   {const}
+ operator<<(ostream&, const Item&): ostream&   {friend}

Menu
- maxSize: int
- items: Item**
+ Menu(int)
+ ~Menu()
+ operator+(Item*): Menu&   <- implement
+ operator<<(ostream&, const Menu&): ostream&   {friend}
```

Differences from the partial UML in the exam paper:

- `operator<<` is not drawn, but the exam paper says every class has one.
- `operator+` returns `Menu&`. The exam paper shows no return type. `Menu&` lets the calls chain.
- `getPrice()` returns `float` here. The exam paper shows `string`, which is probably a mistake, because `price` is a `float`.
- The exam paper's `Menu` also has `findFirst`, `findLast`, `averagePrice`, `numberAbove`, `numberBelow`, `numberEqual`. This question does not use them.

Given code you will use

- `Item`'s `operator<<` prints `<name> (<type>) R<price>` with no newline, for example `Garlic bread (S) R45.5`. The type is `'S'` (starter), `'M'` (main) or `'D'` (dessert).
- `Menu(int maxSize)` makes an array of `maxSize` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.
- `~Menu()` deletes every item in the array and then the array.
- `Menu`'s `operator<<` prints `Menu (max <maxSize>):` and then one line per slot: two spaces, then `[`, the index, `]` and one space, then the item, or `empty` for a `NULL` slot.

What `operator+(Item* item)` must do

1. Find the first slot (the lowest index) that is `NULL`. Store `item` there.
2. If no slot is `NULL` (the menu is full), delete `item`. The menu does not change.

3. In both cases, return the menu itself (`*this`).

More rules:

- Store the item in one slot only. After storing it, stop looking.
- The menu owns every item given to it. The caller never deletes an item it passed to `operator+`. If the menu is full and you do not delete the item, nobody deletes it, and valgrind reports a leak.
- The return type is `Menu&`, a reference. Return `*this`, not a copy.

The given `operator<<` functions and the `ExpectedOutput` block apply to this question only. Another question in this bank may print the same class in a different way.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`. `-Wall -Werror` turns every warning into an error.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A2-05. Operator overloading: `Menu::operator[]` returns `Item*&` (100)

For this question, implement the subscript operator `operator[]` of the `Menu` class in the file `A_2.cpp`. It gives the caller access to one slot of the `items` array. The return type is a reference to the pointer in that slot. So the caller can read the slot, and the caller can store a new pointer in it.

UML (only the classes this question needs)

```
Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ getType(): char   {const}
+ getName(): string   {const}
+ getPrice(): float   {const}
+ operator<<(ostream&, const Item&): ostream&   {friend}

Menu
- maxSize: int
- items: Item**
+ Menu(int)
+ ~Menu()
+ operator+(Item*): Menu&
+ operator[](int): Item*&   <- implement
+ operator<<(ostream&, const Menu&): ostream&   {friend}
```

Differences from the partial UML in the exam paper:

- `Menu::operator[]` (the target) is not in the exam paper. It uses the same skill as the exam's operators: an operator that returns a reference.
- `operator<<` is not drawn, but the exam paper says every class has one.
- `operator+` returns `Menu&`. The exam paper shows no return type.
- `getPrice()` returns `float` here. The exam paper shows `string`, which is probably a mistake, because `price` is a `float`.
- The exam paper's `Menu` also has `findFirst`, `findLast`, `averagePrice`, `numberAbove`, `numberBelow`, `numberEqual`. This question does not use them.

Given code you will use

- `Item`'s `operator<<` prints `<name> (<type>) R<price>` with no newline, for example `Garlic bread (S) R45.5`. The type is `'S'` (starter), `'M'` (main) or `'D'` (dessert).
- `Menu(int maxSize)` makes an array of `maxSize` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.
- `operator+(Item*)` puts the item in the first `NULL` slot and returns the menu, so `menu + a + b;` works. If the menu is full, it deletes the item. The menu owns its items and deletes them in `~Menu()`. Nothing in this question removes items, so the filled slots are always at the front.
- `Menu`'s `operator<<` prints `Menu (max <maxSize>):` and then one line per slot: two spaces, then `[`, the index, `]` and one space, then the item, or `empty` for a `NULL` slot.

What `operator[](int index)` must do

Return the slot `items[index]` itself, as an `Item*&`.

- `menu[i]` then reads the pointer in slot `i`. It can be `NULL`.
- `menu[i] = p;` stores the pointer `p` in slot `i` of the array.

More rules:

- The index is always between 0 and `maxSize - 1` in this question. Do not check it, and do not throw.
- Do not make a new pointer or a new item. Do not delete anything.
- Return a reference to the array element, not to a local variable. If you return a reference to a local variable, g++ gives a `-Wreturn-local-addr` warning, and `-Werror` makes it an error.
- The caller is responsible for the old item when it overwrites a slot. `main.cpp` deletes it first (`delete menu[0];`).

The given `operator<<` functions and the `ExpectedOutput` block apply to this question only. Another question in this bank may print the same class in a different way.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`. `-Wall -Werror` turns every warning into an error.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A2-06. Operator overloading: `Item::operator==` and `Item::operator<` (100)

For this question, implement two comparison operators of the `Item` class in the file `A_2.cpp`: `operator==` and `operator<`. Both are `const` member functions.

UML (only the classes this question needs)

```
Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ getType(): char   {const}
+ getName(): string   {const}
+ getPrice(): float   {const}
+ operator==(const Item&): bool   {const}   <- implement
+ operator<(const Item&): bool   {const}   <- implement
+ operator<<(ostream&, const Item&): ostream&   {friend}
```

Differences from the partial UML in the exam paper:

- `Item::operator==` and `Item::operator<` (the targets) are not in the exam paper. They are operators written as member functions, like the exam's `Menu::operator+`.
- `operator<<` is not drawn, but the exam paper says every class has one.
- `getPrice()` returns `float` here. The exam paper shows `string`, which is probably a mistake, because `price` is a `float`.

Given code you will use

- `Item`'s `operator<<` prints `<name> (<type>) R<price>` with no newline, for example `Garlic bread (S) R45.5`. The type is `'S'` (starter), `'M'` (main) or `'D'` (dessert).

What the two operators must do

1. `a == b` is `true` when the two items have the same type, the same name and the same price. Otherwise it is `false`.
2. `a < b` is `true` when the price of `a` is less than the price of `b`. Only the price decides. If the prices are equal, the result is `false`.

More rules:

- Both functions are `const`: write `const` after the parameter list, as in `Item.h`. They must not change either item.
- The parameter is `const Item& other`. Compare the members of the object itself (the left operand) with the members of `other` (the right operand).
- The prices in this question have an exact `float` value (for example `95.5`), so `==` on two prices is safe here.

The given `operator<<` functions and the `ExpectedOutput` block apply to this question only. Another question in this bank may print the same class in a different way.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`. `-Wall -Werror` turns every warning into an error.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Hints: `HINTS.md` in this folder.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A2-07. Operator overloading: `Menu::operator+=` grows the array when the menu is full (100)

For this question, implement the member operator `operator+=` of the `Menu` class in the file `A_2.cpp`. Like the given `operator+`, it adds one item to the menu. Unlike `operator+`, it never deletes the item: when the menu is full, it makes the array one slot bigger.

UML (only the classes this question needs)

```
Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ getType(): char   {const}
+ getName(): string   {const}
+ getPrice(): float   {const}
+ operator<<(ostream&, const Item&): ostream&   {friend}

Menu
- maxSize: int
- items: Item**
+ Menu(int)
+ ~Menu()
+ operator+(Item*): Menu&
+ operator+=(Item*): Menu&   <- implement
+ operator<<(ostream&, const Menu&): ostream&   {friend}
```

Differences from the partial UML in the exam paper:

- `Menu::operator+=` (the target) is not in the exam paper. Like the exam's `operator+`, it manages the `Item**` array.
- `operator<<` is not drawn, but the exam paper says every class has one.
- `operator+` returns `Menu&`. The exam paper shows no return type.
- `getPrice()` returns `float` here. The exam paper shows `string`, which is probably a mistake, because `price` is a `float`.
- The exam paper's `Menu` also has `findFirst`, `findLast`, `averagePrice`, `numberAbove`, `numberBelow`, `numberEqual`. This question does not use them.

Given code you will use

- `Item`'s `operator<<` prints `<name> (<type>) R<price>` with no newline, for example `Garlic bread (S) R45.5`. The type is `'S'` (starter), `'M'` (main) or `'D'` (dessert).
- `Menu(int maxSize)` makes an array of `maxSize` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.
- `operator+(Item*)` puts the item in the first `NULL` slot and returns the menu, so `menu + a + b;` works. If the menu is full, it deletes the item. The menu owns its items and deletes them in `~Menu()`. Nothing in this question removes items, so the filled slots are always at the front.

- `Menu`'s `operator<<` prints `Menu (max <maxSize>):` and then one line per slot: two spaces, then `[`, the index, `]` and one space, then the item, or `empty` for a `NULL` slot.
- `~Menu()` deletes the items in slots 0 to `maxSize - 1`, then the array. It uses the current value of `maxSize`.

What `operator+=(Item* item)` must do

1. If the menu has a `NULL` slot, store `item` in the first one (the lowest index) and stop.
2. Otherwise (the menu is full):
3. Make a new array of `maxSize + 1` pointers.
4. Copy the pointers from the old array into slots 0 to `maxSize - 1` of the new array. Do this before you change `maxSize`.
5. Store `item` in the last slot, index `maxSize`.
6. Delete the old array with `delete[]`. Do not delete the items: the new array points to them now.
7. Make `items` point to the new array and add 1 to `maxSize`.
8. In both cases, return the menu itself (`*this`).

More rules:

- The menu owns every item given to it. `operator+=` never deletes the item it is given.
- The new array grows by exactly one slot.
- Copy the pointers, not the items. Do not call `new Item(...)`.

The given `operator<<` functions and the `ExpectedOutput` block apply to this question only. Another question in this bank may print the same class in a different way.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`. `-Wall -Werror` turns every warning into an error.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A2-08. Operator overloading: `Menu::operator-(string)` removes an item by name (100)

For this question, implement the member operator `operator-` of the `Menu` class in the file `A_2.cpp`. It removes the first item with a given name and keeps the filled slots at the front of the array.

UML (only the classes this question needs)

```
Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ getType(): char   {const}
+ getName(): string   {const}
+ getPrice(): float   {const}
+ operator<<(ostream&, const Item&): ostream&   {friend}

Menu
- maxSize: int
- items: Item**
+ Menu(int)
+ ~Menu()
+ operator+(Item*): Menu&
+ operator-(string): Menu&   <- implement
+ operator<<(ostream&, const Menu&): ostream&   {friend}
```

Differences from the partial UML in the exam paper:

- `Menu::operator-` (the target) is not in the exam paper. Like the exam's `operator+`, it manages the `Item**` array.
- `operator<<` is not drawn, but the exam paper says every class has one.
- `operator+` returns `Menu&`. The exam paper shows no return type.
- `getPrice()` returns `float` here. The exam paper shows `string`, which is probably a mistake, because `price` is a `float`.
- The exam paper's `Menu` also has `findFirst`, `findLast`, `averagePrice`, `numberAbove`, `numberBelow`, `numberEqual`. This question does not use them.

Given code you will use

- `Item`'s `operator<<` prints `<name> (<type>) R<price>` with no newline, for example `Garlic bread (S) R45.5`. The type is `'S'` (starter), `'M'` (main) or `'D'` (dessert).
- `Menu(int maxSize)` makes an array of `maxSize` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.
- `operator+(Item*)` puts the item in the first `NULL` slot and returns the menu. If the menu is full, it deletes the item. The menu owns its items and deletes them in `~Menu()`.
- `Menu`'s `operator<<` prints `Menu (max <maxSize>):` and then one line per slot: two spaces, then `[`, the index, `]` and one space, then the item, or `empty` for a `NULL` slot.

What `operator-(string name)` must do

1. Find the first item (the lowest index) whose name is equal to `name`.
2. If there is one:
3. Delete that item.
4. Move every pointer after it one slot towards index 0, so that there is no empty slot between two items.
5. Set the last slot, `items[maxSize - 1]`, to `NULL`.
6. If no item has that name, do not change the menu.
7. In both cases, return the menu itself (`*this`).

Example: a menu of size 4 holds `Garlic bread`, `Beef burger`, `Malva pudding` and one empty slot. After `menu - "Beef burger";` it holds `Garlic bread`, `Malva pudding` and two empty slots.

More rules:

- Remove one item only. If a later item has the same name, it stays.
- The names must match exactly (`==` on two `string`s).
- Do not call `getName()` on a `NULL` slot.
- Do not read `items[maxSize]`: the valid indexes are 0 to `maxSize - 1`.

The given `operator<<` functions and the `ExpectedOutput` block apply to this question only. Another question in this bank may print the same class in a different way.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`. `-Wall -Werror` turns every warning into an error.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

A_3 Assignment operator

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A3-01. Assignment operator: `Item::operator=` (100)

Implement the assignment operator of the `Item` class in `A_3.cpp`. After `a = b;`, item `a` has the same type, name and price as item `b`.

```
Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ operator=(const Item&): Item&     <- implement
+ getType(): char
+ getName(): string
+ getPrice(): float
```

What the function must do

- Copy all three members: `type`, `name` and `price`.
- Assigning an item to itself (`a = a;`) must leave it unchanged.
- Return `*this`, so that `a = b = c;` works.
- `operator<<` for `Item` prints `<name> (<type>) R<price>`, for example `Soup (S) R50`.

Rules

- Write only this one function in `A_3.cpp`. Do not change the other files: they are replaced when marked.
- The program must compile with `-Wall -Werror`, print exactly the output at the end of `main.cpp`, and have no memory errors or leaks under valgrind.

Build and upload

In the `A_3` folder, run `make A_3`. Upload only `A_3.cpp`.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A3-02. Assignment operator: `Menu::operator=` (100)

Implement the assignment operator of the `Menu` class in `A_3.cpp`. After `a = b;`, menu `a` must be a deep copy of menu `b`: the same `maxSize`, and its own copy of every item, in the same slots.

```
Menu
- maxSize: int
- items: Item**
+ Menu(int)
+ ~Menu()
+ operator=(const Menu&): Menu&     <- implement
+ operator+(Item*): Menu&
+ findFirst(string): Item
+ findLast(string): Item
+ averagePrice(string): float
+ numberAbove(string, float): int
+ numberBelow(string, float): int
+ numberEqual(string, float): int

Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ getType(): char
+ getName(): string
+ getPrice(): float
```

What the function must do

- `maxSize` is the length of the `items` array. An empty slot holds `NULL`.
- The menu owns its items: `~Menu()` deletes every item and then the array.
- `operator+(Item*)` puts the item in the first `NULL` slot. If the menu is full, it deletes the item.
- `Item` has a copy constructor, so `new Item(*p)` makes a new item with the same type, name and price as `*p`.
- `operator<<` prints `Menu (max N):` and then one line per slot: `[i] <item>` or `[i] empty`.
- After `a = b;`, `a` has the same `maxSize` as `b`. Slot `i` of `a` holds a new copy of the item in slot `i` of `b`, or `NULL` if that slot of `b` is `NULL`.
- The items and the array that `a` held before must be freed.
- Changing `b` afterwards (for example adding an item) must not change `a`.
- `a = a;` must leave `a` unchanged. `a = b = c;` must work.

Rules

- Write only this one function in `A_3.cpp`. Do not change the other files: they are replaced when marked.
- The program must compile with `-Wall -Werror`, print exactly the output at the end of `main.cpp`, and have no memory errors or leaks under valgrind.

Build and upload

In the `A_3` folder, run `make A_3`. Upload only `A_3.cpp`.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A3-03. Copy constructor: `Menu::Menu(const Menu&)` (100)

The copy constructor is not in the published UML; this question practises the same deep copy as `operator=`. Implement the copy constructor of the `Menu` class in `A_3.cpp`. The new menu must be a deep copy of `other`.

```
Menu
- maxSize: int
- items: Item**
+ Menu(int)
+ Menu(const Menu&)     <- implement
+ ~Menu()
+ operator+(Item*): Menu&
+ findFirst(string): Item
+ findLast(string): Item
+ averagePrice(string): float
+ numberAbove(string, float): int
+ numberBelow(string, float): int
+ numberEqual(string, float): int

Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ getType(): char
+ getName(): string
+ getPrice(): float
```

What the function must do

- `maxSize` is the length of the `items` array. An empty slot holds `NULL`.
- The menu owns its items: `~Menu()` deletes every item and then the array.
- `operator+(Item*)` puts the item in the first `NULL` slot. If the menu is full, it deletes the item.
- `Item` has a copy constructor, so `new Item(*p)` makes a new item with the same type, name and price as `*p`.
- `operator<<` prints `Menu (max N):` and then one line per slot: `[i] <item>` or `[i] empty`.
- The new menu has the same `maxSize` as `other`. Slot `i` holds a new copy of `other`'s item in slot `i`, or `NULL`.
- Changing `other` afterwards must not change the copy.
- The copy constructor runs for `Menu b(a);`, for `Menu c = b;` and when a `Menu` is passed by value.

Rules

- Write only this one function in `A_3.cpp`. Do not change the other files: they are replaced when marked.
- The program must compile with `-Wall -Werror`, print exactly the output at the end of `main.cpp`, and have no memory errors or leaks under valgrind.

Build and upload

In the `A_3` folder, run `make A_3`. Upload only `A_3.cpp`.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A3-04. Assignment operator: `Menu::operator=` that keeps its own size (100)

Implement the assignment operator of the `Menu` class in `A_3.cpp`. In this version the menu on the left keeps its own `maxSize`: its array is not replaced.

```
Menu
- maxSize: int
- items: Item**
+ Menu(int)
+ ~Menu()
+ operator=(const Menu&): Menu&     <- implement
+ operator+(Item*): Menu&
+ findFirst(string): Item
+ findLast(string): Item
+ averagePrice(string): float
+ numberAbove(string, float): int
+ numberBelow(string, float): int
+ numberEqual(string, float): int

Item
- type: char
- name: string
- price: float
+ Item(char, string, float)
+ Item(const Item&)
+ ~Item()
+ getType(): char
+ getName(): string
+ getPrice(): float
```

What the function must do

- `maxSize` is the length of the `items` array. An empty slot holds `NULL`.
- The menu owns its items: `~Menu()` deletes every item and then the array.
- `operator+(Item*)` puts the item in the first `NULL` slot. If the menu is full, it deletes the item.
- `Item` has a copy constructor, so `new Item(*p)` makes a new item with the same type, name and price as `*p`.
- `operator<<` prints `Menu (max N):` and then one line per slot: `[i] <item>` or `[i] empty`.
- After `a = b;`, `a` keeps its own `maxSize`. All of `a`'s old items are deleted.
- For every slot number that both menus have, slot `i` of `a` gets a new copy of the item in slot `i` of `b` (or stays `NULL` if that slot of `b` is `NULL`).
- If `b` has more slots than `a`, the items in `b`'s extra slots are not copied. If `a` has more slots than `b`, `a`'s extra slots are `NULL`.
- `a = a;` must leave `a` unchanged. `a = b = c;` must work.

Rules

- Write only this one function in `A_3.cpp`. Do not change the other files: they are replaced when marked.

- The program must compile with `-Wall -Werror`, print exactly the output at the end of `main.cpp`, and have no memory errors or leaks under valgrind.

Build and upload

In the `A_3` folder, run `make A_3`. Upload only `A_3.cpp`.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

A3-05. Assignment operator: `Binary::operator=` (a `bool*` array) (100)

`Binary` is from Section B of the scenario. Its `operator=` is not in the published UML; this question practises the four steps of `operator=` on a plain array. Implement the assignment operator of the `Binary` class in `A_3.cpp`.

```
Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator=(const Binary&): Binary&     <- implement
+ operator[](int): bool&
+ operator[](int): const bool&
+ getSize(): int
```

What the function must do

- `array` holds `size` bools. `Binary("1101")` makes `true, true, false, true`.
- After `a = b;`, `a` has the same `size` as `b` and its own new array with the same bits.
- `a`'s old array must be freed.
- Changing `b` afterwards (for example `b[0] = false;`) must not change `a`.
- `a = a;` must leave `a` unchanged. `a = b = c;` must work.
- `operator<<` prints the bits as `0` and `1` characters.

Rules

- Write only this one function in `A_3.cpp`. Do not change the other files: they are replaced when marked.
- The program must compile with `-Wall -Werror`, print exactly the output at the end of `main.cpp`, and have no memory errors or leaks under valgrind.

Build and upload

In the `A_3` folder, run `make A_3`. Upload only `A_3.cpp`.

B_1 Recursion

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B1-01. Recursion: `Binary::toDecimal` (100)

For this question, implement the function `toDecimal` of the class `Binary` **using recursion**, in the file `B_1.cpp`. All the other functions of `Binary` are given in `Binary.cpp`.

UML (only the class this question needs)

```
Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ toDecimal(int): int   <- implement
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `toDecimal` (the target), `operator<<` (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- `Binary(string)` sets `size` to the length of the string and creates `array` with `new bool[size]`. Each `'1'` becomes `true`. Every other character becomes `false`.
- `array[0]` holds the **leftmost** character of the string. In this question it is the most significant bit.
- `operator<<` prints the bits from index 0 to the end, for example `1011`. It prints no newline.
- `operator=` is private and not defined in this question. Do not assign one `Binary` to another.
- Inside `Binary` you can use `array` and `size` directly.

What `toDecimal(index)` must do

It returns the decimal (base 10) value of the bits from index 0 up to **and including** `index`, read as an unsigned binary number.

- Example: for `1011`, `toDecimal(3)` returns 11 (the whole string), `toDecimal(1)` returns 2 (the bits `10`) and `toDecimal(0)` returns 1 (the bit `1`).
- To get the value of the whole string, call `toDecimal(getSize() - 1)`.
- Invalid input: if `index` is less than 0, or `index` is greater than or equal to `size`, return 0.
- Do not change any bit.
- The tests use at most 31 bits, so the value fits in an `int`.

Hint: the value of the bits up to `index` is two times the value of the bits up to `index - 1`, plus the bit at `index` (1 or 0). When `index` reaches -1, no bits are left and the value is 0.

Rules

- You must use recursion. **No loops (`for`, `while`, `do`) and no helper functions. Using either scores 0.** The function must call itself. It may call the given functions. It may not call any other function that you write.
- Your code must compile with `-std=c++98 -Wall -Werror` and must not leak memory.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_1` folder.
2. Run `make B_1`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example with a segmentation fault), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B1-02. Recursion: `Binary::countOnes` (100)

For this question, implement the function `countOnes` of the class `Binary` **using recursion**, in the file `B_1.cpp`. All the other functions of `Binary` are given in `Binary.cpp`.

UML (only the class this question needs)

```
Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ countOnes(int): int   <- implement
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `countOnes` (the target), `operator<<` (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- `Binary(string)` sets `size` to the length of the string and creates `array` with `new bool[size]`. Each `'1'` becomes `true`. Every other character becomes `false`.
- `array[0]` holds the leftmost character of the string.
- `operator<<` prints the bits from index 0 to the end, for example `10110`. It prints no newline.
- `operator=` is private and not defined in this question. Do not assign one `Binary` to another.
- Inside `Binary` you can use `array` and `size` directly.

What `countOnes(index)` must do

It returns the number of `true` bits (1 bits) from `index` to the end of the array.

- Example: for `10110`, `countOnes(0)` returns 3 and `countOnes(2)` returns 2 (the bits `110`).
- If `index` is less than 0, or equal to or greater than `size`, return 0.
- Do not change any bit.

Rules

- You must use recursion. **No loops (`for`, `while`, `do`) and no helper functions. Using either scores 0.** The function must call itself. It may call the given functions. It may not call any other function that you write.
- Your code must compile with `-std=c++98 -Wall -Werror` and must not leak memory.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_1` folder.
2. Run `make B_1`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example with a segmentation fault), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B1-03. Recursion: `Binary::isPalindrome` (100)

For this question, implement the function `isPalindrome` of the class `Binary` **using recursion**, in the file `B_1.cpp`. All the other functions of `Binary` are given in `Binary.cpp`.

UML (only the class this question needs)

```
Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ isPalindrome(int, int): bool   <- implement
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `isPalindrome` (the target), `operator<<` (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- `Binary(string)` sets `size` to the length of the string and creates `array` with `new bool[size]`. Each `'1'` becomes `true`. Every other character becomes `false`.
- `array[0]` holds the leftmost character of the string.
- `operator<<` prints the bits from index 0 to the end, for example `1001`. It prints no newline.
- `operator=` is private and not defined in this question. Do not assign one `Binary` to another.
- Inside `Binary` you can use `array` and `size` directly.

What `isPalindrome(start, end)` must do

A palindrome reads the same from left to right and from right to left. The function checks the bits from index `start` to index `end`, both included.

- It returns `true` if the bits in that range form a palindrome, and `false` if they do not.
- Examples: `1001` with `(0, 3)` gives `true`. `1011` with `(0, 3)` gives `false`. `1011` with `(0, 2)` checks `101` and gives `true`.
- A range with one bit (`start` equals `end`) or with no bits (`start` is greater than `end`) is a palindrome: return `true`.
- Invalid input: if `start` is less than 0, or `end` is greater than or equal to `size`, return `false`. Check this before anything else.
- Do not change any bit.

Rules

- You must use recursion. **No loops (`for`, `while`, `do`) and no helper functions. Using either scores 0.** The function must call itself. It may call the given functions. It may not call any other

function that you write.
- Your code must compile with `-std=c++98 -Wall -Werror` and must not leak memory.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_1` folder.
2. Run `make B_1`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example with a segmentation fault), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B1-04. Recursion: `BinaryStrings::countNegative` (100)

For this question, implement the function `countNegative` of the class `BinaryStrings` **using recursion**, in the file `B_1.cpp`. All the other functions are given in the other files.

UML (only the classes this question needs)

```
BinaryStrings
- head: Node<Binary>*
- size: int
+ BinaryStrings()
+ ~BinaryStrings()
+ prepend(const Binary&): void
+ getHead(): Node<Binary>*&
+ calculatePos(): int
+ countNegative(Node<Binary>*): int   <- implement
+ operator<<(ostream&, const BinaryStrings&): ostream&   {friend}

Node<T>
- data: T
- nextNode: Node<T>*
+ Node(T)
+ ~Node()
+ getData(): T&
+ next(): Node<T>*&

Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `countNegative` (the target), the `operator<<` functions (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- `getHead()` returns the first node of the list, or `NULL` for an empty list.
- `next()` returns the next node, or `NULL` after the last node.
- `getData()` returns a reference to the `Binary` stored in the node. So `curr->getData()[0]` is bit 0 of that string, and `curr->getData().getSize()` is its number of bits.
- `Binary(string)`: each `'1'` becomes `true`; every other character becomes `false`. `array[0]` is the leftmost bit.
- `operator<<` for `BinaryStrings` prints `Empty`, or the strings joined by `->`, for example `100 -> 0011 -> 1101`. It prints no newline.

What `countNegative(curr)` must do

In this question bit 0 is a sign bit: 1 means negative, 0 means not negative. A binary string is **negative** when it has at least one bit and its bit 0 is `true` (1).

- It returns the number of negative strings from the node `curr` to the end of the list.
- Example: for the list `100 -> 0011 -> 1101 -> 0101`, `countNegative(getHead())` returns 2.
- If `curr` is `NULL`, return 0. (This also covers an empty list.)
- An empty string (0 bits) has no bit 0. It is not negative. Do not read bit 0 of an empty string.
- Do not change the list or any bit.

Rules

- You must use recursion. **No loops (`for`, `while`, `do`) and no helper functions. Using either scores 0.** The function must call itself. It may call the given functions. It may not call any other function that you write.
- Your code must compile with `-std=c++98 -Wall -Werror` and must not leak memory.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_1` folder.
2. Run `make B_1`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example with a segmentation fault), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B1-05. Recursion: `BinaryStrings::calculatePos` with a node and a bit index (100)

For this question, implement the function `calculatePos` of the class `BinaryStrings` **using recursion**, in the file `B_1.cpp`. All the other functions are given in the other files.

This question changes the UML on purpose. The partial UML in the exam paper shows `calculatePos(): int` with no parameters. A function with no parameters has nothing to change from one call to the next, so it cannot walk the list by recursion. Here `calculatePos` takes the node to start from and the bit to check. The no-parameter version is not in this question. If the exam keeps it and allows a helper, this same body goes in the helper, and `calculatePos()` calls it once.

UML (only the classes this question needs)

```
BinaryStrings
- head: Node<Binary>*
- size: int
+ BinaryStrings()
+ ~BinaryStrings()
+ prepend(const Binary&): void
+ getHead(): Node<Binary>*&
+ calculatePos(Node<Binary>*, int): int   <- implement
+ operator<<(ostream&, const BinaryStrings&): ostream&   {friend}

Node<T>
- data: T
- nextNode: Node<T>*
+ Node(T)
+ ~Node()
+ getData(): T&
+ next(): Node<T>*&

Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: the two parameters of `calculatePos` (see above), the `operator<<` functions (not drawn; the exam paper says every class has one), and the `{const}` markers. The exam paper does not say what "Pos" means. In this question it means "positive", as defined below. The real exam's complete UML and question text decide.

What is given

- `getHead()` returns the first node of the list, or `NULL` for an empty list.
- `next()` returns the next node, or `NULL` after the last node.

- `getData()` returns a reference to the `Binary` stored in the node. So `curr->getData()[i]` is bit `i` of that string, and `curr->getData().getSize()` is its number of bits.
- `Binary(string)`: each `'1'` becomes `true`; every other character becomes `false`. `array[0]` is the leftmost bit.
- `operator<<` for `BinaryStrings` prints `Empty`, or the strings joined by `->`. It prints no newline.

What `calculatePos(curr, index)` must do

Bit 0 is a sign bit. A binary string is **positive** when bit 0 is `false` (0) **and** at least one other bit is `true` (1). So `0011` is positive. `000` (zero), `1101` (negative), `0`, `1` and the empty string are not positive.

- It returns the number of positive strings from the node `curr` to the end of the list.
- `index` is the bit of `curr` that this call checks. The first call is `calculatePos(getHead(), 1)`. The bits of `curr` before `index` are not checked again.
- When the function moves to the next node, it starts again at bit 1 of that node.
- Example: for `000 -> 0011 -> 1101 -> 0101`, `calculatePos(getHead(), 1)` returns 2.
- If `curr` is `NULL`, return 0. (This also covers an empty list.)
- Invalid input: if `index` is less than 1, return 0.
- If `index` is greater than or equal to the number of bits of `curr`'s string, the check has passed the last bit without finding a 1. This string is not positive. Continue with the next node, at bit 1.
- If bit `index` is `true` and bit 0 is `false`, count this string once and continue with the next node, at bit 1.
- If bit `index` is `false`, check the next bit of the same string: `index + 1`.
- Never read a bit outside a string. An empty string has no bit 0.
- Do not change the list or any bit.

Rules

- You must use recursion. **No loops (`for`, `while`, `do`) and no helper functions. Using either scores 0.** The function must call itself. It may call the given functions. It may not call any other function that you write.
- Your code must compile with `-std=c++98 -Wall -Werror` and must not leak memory.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_1` folder.
2. Run `make B_1`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example with a segmentation fault), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B1-06. Recursion: `BinaryStrings::printReverse` (100)

For this question, implement the function `printReverse` of the class `BinaryStrings` **using recursion**, in the file `B_1.cpp`. All the other functions are given in the other files.

UML (only the classes this question needs)

```
BinaryStrings
- head: Node<Binary>*
- size: int
+ BinaryStrings()
+ ~BinaryStrings()
+ prepend(const Binary&): void
+ getHead(): Node<Binary>*&
+ calculatePos(): int
+ printReverse(Node<Binary>*): void   <- implement
+ operator<<(ostream&, const BinaryStrings&): ostream&   {friend}

Node<T>
- data: T
- nextNode: Node<T>*
+ Node(T)
+ ~Node()
+ getData(): T&
+ next(): Node<T>*&

Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `printReverse` (the target), the `operator<<` functions (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- `getHead()` returns the first node of the list, or `NULL` for an empty list.
- `next()` returns the next node, or `NULL` after the last node.
- `getData()` returns a reference to the `Binary` stored in the node.
- `operator<<` for a `Binary` prints its bits, for example `0011`. An empty string prints nothing. It prints no newline.
- `operator<<` for `BinaryStrings` prints `Empty`, or the strings joined by `->`, for example `000 -> 0011 -> 1101`.

What `printReverse(curr)` must do

It prints the strings from the **last** node of the list back to the node `curr`.

- Print each string with `cout`, followed by `endl`. So each string is on its own line, and nothing else is printed. An empty string gives an empty line.
- Example: for the list `000 -> 0011 -> 1101 -> 0101`, `printReverse(getHead())` prints `0101`, `1101`, `0011` and `000`, each on its own line.
- If `curr` is `NULL`, print nothing. (This also covers an empty list.)
- Do not change the list.

Rules

- You must use recursion. **No loops (`for`, `while`, `do`) and no helper functions. Using either scores 0.** The function must call itself. It may call the given functions. It may not call any other function that you write.
- Your code must compile with `-std=c++98 -Wall -Werror` and must not leak memory.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_1` folder.
2. Run `make B_1`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example with a segmentation fault), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B1-07. Recursion: `Binary::setBits` (100)

For this question, implement the function `setBits` of the class `Binary` **using recursion**, in the file `B_1.cpp`. The given constructor `Binary(string)` calls your function to fill the bool array. All the other functions of `Binary` are given in `Binary.cpp`.

UML (only the class this question needs)

```
Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ setBits(string, int): void   <- implement
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `setBits` (the target), `operator<<` (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- The constructor (in `Binary.cpp`): `cpp Binary::Binary(string bits) { size = (int)bits.length(); array = new bool[size]; setBits(bits, 0); }` So every `Binary` object in `main.cpp` is filled by your `setBits`. The array already exists when `setBits` is called. Do not create or delete it.
- `operator<<` prints the bits from index 0 to the end, for example `1011`. It prints no newline.
- `operator=` is private and not defined in this question. Do not assign one `Binary` to another.
- Inside `Binary` you can use `array` and `size` directly.

What `setBits(bits, index)` must do

It sets `array[index]`, `array[index + 1]`, and so on, from the characters at the **same positions** in `bits`.

- A `'1'` gives `true`. Every other character gives `false`.
- It stops when `index` reaches `size` or `bits.length()`, the smaller of the two. Bits after that keep their old values.
- Bits before `index` keep their old values.
- Examples: for a `Binary` that holds `0000`, `setBits("11", 0)` gives `1100`. Then `setBits("0110", 2)` gives `1110` (only bits 2 and 3 change). `setBits("111111", 0)` gives `1111` (the string is longer than the array).
- Invalid input: if `index` is less than 0, change nothing.
- `bits.length()` is unsigned. Under `-Wall -Werror`, comparing it with an `int` stops the build. Cast it: `(int)bits.length()`.

Rules

- You must use recursion. **No loops (`for`, `while`, `do`) and no helper functions. Using either scores 0.** The function must call itself. It may call the given functions. It may not call any other function that you write.
- Your code must compile with `-std=c++98 -Wall -Werror` and must not leak memory.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_1` folder.
2. Run `make B_1`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example with a segmentation fault), the mark is 0.

B_2 Linked lists

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B2-01. Linked lists: `BinaryStrings::append` (100)

For this question, implement the function `append` of the class `BinaryStrings` in the file `B_2.cpp`. It adds a binary string at the **end** of the list. All the other functions are given in the other files.

UML (only the classes this question needs)

```
BinaryStrings
- head: Node<Binary>*
- size: int
+ BinaryStrings()
+ ~BinaryStrings()
+ prepend(const Binary&): void
+ getHead(): Node<Binary>*&
+ calculatePos(): int
+ getSize(): int
+ append(const Binary&): void   <- implement
+ operator<<(ostream&, const BinaryStrings&): ostream&   {friend}

Node<T>
- data: T
- nextNode: Node<T>*
+ Node(T)
+ ~Node()
+ getData(): T&
+ next(): Node<T>*&

Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `append` (the target), `BinaryStrings::getSize` (added so that the tests can print `size`), the `operator<<` functions (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores its own **copy** of the data (`data: T`, not `T*`). `new Node<Binary>(b)` makes a node that holds a copy of `b`. The copy is made by the `Binary` copy constructor.
- `~Node()` is empty. It does **not** delete the next node. When a node is deleted, the `Binary` inside it is destroyed too, and that frees its bool array. So `delete curr;` is all that is needed to free one node and its string.

- `next()` returns a reference to the pointer to the next node (the link), so `curr->next() = ...;` changes the link.
- `getData()` returns a reference to the `Binary` in the node. `curr->getData()[i]` is bit `i` of that string, and `curr->getData().getSize()` is its number of bits.
- `prepend(b)` puts a new node at the front and adds 1 to `size`.
- The destructor `~BinaryStrings()` deletes every node.
- `size` must always equal the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty`, or the strings joined by `->`, for example `000 -> 0011 -> 1101`. It prints no newline. An empty string prints nothing. For example, a list of `0101`, an empty string and `11` prints as follows (two spaces between the arrows):

```
text 0101 -> -> 11
```

- `Binary(string)`: each `'1'` becomes `true`; every other character becomes `false`. `array[0]` is the leftmost bit.
- Inside `BinaryStrings` you can use `head` and `size` directly.
- `operator=` of `Binary`, and the copy constructor and `operator=` of `BinaryStrings`, are private and not defined in this question. Do not use them.
- In examples, a call such as `append("0101")` is short for `append(Binary("0101"))`.

What `append(b)` must do

- Create a new node that holds a copy of `b`, and link it after the last node.
- If the list is empty, the new node becomes the head.
- The new node's `next()` is `NULL` (the `Node` constructor already sets it).
- Add 1 to `size`.
- Example: `prepend("000")`, then `append("0101")` and `append("11")`, gives `000 -> 0101 -> 11`.
- A later change to the caller's `Binary` must not change the list. (The node's copy makes this work.)

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.
- You may not change any header, so write everything in `B_2.cpp`. A small helper function in `B_2.cpp` is allowed. The ban on helpers applies only to the B_1 recursion questions. Make it `static` so that its name stays inside `B_2.cpp`. In the exam, follow the question text.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example with a segmentation fault), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B2-02. Linked lists: `BinaryStrings::insertAt` (100)

For this question, implement the function `insertAt` of the class `BinaryStrings` in the file `B_2.cpp`. It inserts a binary string at a given position. All the other functions are given in the other files.

UML (only the classes this question needs)

```
BinaryStrings
- head: Node<Binary>*
- size: int
+ BinaryStrings()
+ ~BinaryStrings()
+ prepend(const Binary&): void
+ getHead(): Node<Binary>*&
+ calculatePos(): int
+ getSize(): int
+ insertAt(const Binary&, int): void   <- implement
+ operator<<(ostream&, const BinaryStrings&): ostream&   {friend}

Node<T>
- data: T
- nextNode: Node<T>*
+ Node(T)
+ ~Node()
+ getData(): T&
+ next(): Node<T>*&

Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `insertAt` (the target), `BinaryStrings::getSize` (added so that the tests can print `size`), the `operator<<` functions (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores its own **copy** of the data (`data: T`, not `T*`). `new Node<Binary>(b)` makes a node that holds a copy of `b`. The copy is made by the `Binary` copy constructor.
- `~Node()` is empty. It does **not** delete the next node. When a node is deleted, the `Binary` inside it is destroyed too, and that frees its bool array. So `delete curr;` is all that is needed to free one node and its string.
- `next()` returns a reference to the pointer to the next node (the link), so `curr->next() = ...;` changes the link.
- `getData()` returns a reference to the `Binary` in the node. `curr->getData()[i]` is bit `i` of that string, and `curr->getData().getSize()` is its number of bits.

- `prepend(b)` puts a new node at the front and adds 1 to `size`.
- The destructor `~BinaryStrings()` deletes every node.
- `size` must always equal the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty`, or the strings joined by `->`, for example `000 -> 0011 -> 1101`. It prints no newline. An empty string prints nothing. For example, a list of `0101`, an empty string and `11` prints as follows (two spaces between the arrows):

```
text 0101 -> -> 11
```

- `Binary(string)`: each `'1'` becomes `true`; every other character becomes `false`. `array[0]` is the leftmost bit.
- Inside `BinaryStrings` you can use `head` and `size` directly.
- `operator=` of `Binary`, and the copy constructor and `operator=` of `BinaryStrings`, are private and not defined in this question. Do not use them.
- In examples, a call such as `insertAt("01", 2)` is short for `insertAt(Binary("01"), 2)`.

What `insertAt(b, pos)` must do

Positions count from 0: the head is position 0. These are the rules of the combined insert in the lectures (Lecture 22).

- Create a new node that holds a copy of `b`. After the call, it is the node at position `pos`. The nodes that were at `pos` and after it move one position toward the end of the list.
- If `pos` is 0 or less, insert at the front.
- If the list is empty, the new node is the only node, for any value of `pos`.
- If `pos` is equal to or greater than the number of nodes, append at the end.
- Add 1 to `size` in every case.
- Example: for `000 -> 0011 -> 1101`, `insertAt("01", 2)` gives `000 -> 0011 -> 01 -> 1101`.
- The last node's `next()` must be `NULL`.

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.
- You may not change any header, so write everything in `B_2.cpp`. A small helper function in `B_2.cpp` is allowed. The ban on helpers applies only to the B_1 recursion questions. Make it `static` so that its name stays inside `B_2.cpp`. In the exam, follow the question text.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example with a

segmentation fault), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B2-03. Linked lists: `BinaryStrings::removeAt` (100)

For this question, implement the function `removeAt` of the class `BinaryStrings` in the file `B_2.cpp`. It removes the node at a given position. All the other functions are given in the other files.

UML (only the classes this question needs)

```
BinaryStrings
- head: Node<Binary>*
- size: int
+ BinaryStrings()
+ ~BinaryStrings()
+ prepend(const Binary&): void
+ getHead(): Node<Binary>*&
+ calculatePos(): int
+ getSize(): int
+ removeAt(int): bool   <- implement
+ operator<<(ostream&, const BinaryStrings&): ostream&   {friend}

Node<T>
- data: T
- nextNode: Node<T>*
+ Node(T)
+ ~Node()
+ getData(): T&
+ next(): Node<T>*&

Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `removeAt` (the target), `BinaryStrings::getSize` (added so that the tests can print `size`), the `operator<<` functions (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores its own **copy** of the data (`data: T`, not `T*`). `new Node<Binary>(b)` makes a node that holds a copy of `b`. The copy is made by the `Binary` copy constructor.
- `~Node()` is empty. It does **not** delete the next node. When a node is deleted, the `Binary` inside it is destroyed too, and that frees its bool array. So `delete curr;` is all that is needed to free one node and its string.
- `next()` returns a reference to the pointer to the next node (the link), so `curr->next() = ...;` changes the link.
- `getData()` returns a reference to the `Binary` in the node. `curr->getData()[i]` is bit `i` of that string, and `curr->getData().getSize()` is its number of bits.

- `prepend(b)` puts a new node at the front and adds 1 to `size`.
- The destructor `~BinaryStrings()` deletes every node.
- `size` must always equal the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty`, or the strings joined by `->`, for example `000 -> 0011 -> 1101`. It prints no newline. An empty string prints nothing. For example, a list of `0101`, an empty string and `11` prints as follows (two spaces between the arrows):

```
text 0101 -> -> 11
```

- `Binary(string)`: each `'1'` becomes `true`; every other character becomes `false`. `array[0]` is the leftmost bit.
- Inside `BinaryStrings` you can use `head` and `size` directly.
- `operator=` of `Binary`, and the copy constructor and `operator=` of `BinaryStrings`, are private and not defined in this question. Do not use them.

What `removeAt(pos)` must do

Positions count from 0: the head is position 0.

- If `pos` is a valid position (0 to `size - 1`), unlink that node, delete it, subtract 1 from `size` and return `true`.
- If `pos` is less than 0, or equal to or greater than `size`, change nothing and return `false`. (An empty list has no valid position.)
- Deleting the node frees its `Binary` as well. Do not delete anything else, and do not delete a node twice.
- The nodes before and after the removed node must stay linked. If the last node is removed, the new last node's `next()` must be `NULL`. If the only node is removed, `head` must be `NULL`.
- Example: for `1 -> 000 -> 0011 -> 1101 -> 0101`, `removeAt(2)` returns `true` and gives `1 -> 000 -> 1101 -> 0101`.

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.
- You may not change any header, so write everything in `B_2.cpp`. A small helper function in `B_2.cpp` is allowed. The ban on helpers applies only to the B_1 recursion questions. Make it `static` so that its name stays inside `B_2.cpp`. In the exam, follow the question text.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example with a

segmentation fault), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B2-04. Linked lists: `BinaryStrings::remove` the first equal string (100)

For this question, implement the function `remove` of the class `BinaryStrings` in the file `B_2.cpp`. It removes the first binary string that is equal to a given one. All the other functions are given in the other files.

UML (only the classes this question needs)

```
BinaryStrings
- head: Node<Binary>*
- size: int
+ BinaryStrings()
+ ~BinaryStrings()
+ prepend(const Binary&): void
+ getHead(): Node<Binary>*&
+ calculatePos(): int
+ getSize(): int
+ remove(const Binary&): bool   <- implement
+ operator<<(ostream&, const BinaryStrings&): ostream&   {friend}

Node<T>
- data: T
- nextNode: Node<T>*
+ Node(T)
+ ~Node()
+ getData(): T&
+ next(): Node<T>*&

Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `remove` (the target), `BinaryStrings::getSize` (added so that the tests can print `size`), the `operator<<` functions (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores its own **copy** of the data (`data: T`, not `T*`). `new Node<Binary>(b)` makes a node that holds a copy of `b`. The copy is made by the `Binary` copy constructor.
- `~Node()` is empty. It does **not** delete the next node. When a node is deleted, the `Binary` inside it is destroyed too, and that frees its bool array. So `delete curr;` is all that is needed to free one node and its string.
- `next()` returns a reference to the pointer to the next node (the link), so `curr->next() = ...;` changes the link.

- `getData()` returns a reference to the `Binary` in the node. `curr->getData()[i]` is bit `i` of that string, and `curr->getData().getSize()` is its number of bits.
- `prepend(b)` puts a new node at the front and adds 1 to `size`.
- The destructor `~BinaryStrings()` deletes every node.
- `size` must always equal the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty`, or the strings joined by `->`, for example `000 -> 0011 -> 1101`. It prints no newline. An empty string prints nothing. For example, a list of `0101`, an empty string and `11` prints as follows (two spaces between the arrows):

```
text 0101 -> -> 11
```

- `Binary(string)`: each `'1'` becomes `true`; every other character becomes `false`. `array[0]` is the leftmost bit.
- Inside `BinaryStrings` you can use `head` and `size` directly.
- `operator=` of `Binary`, and the copy constructor and `operator=` of `BinaryStrings`, are private and not defined in this question. Do not use them.
- In examples, a call such as `remove("0011")` is short for `remove(Binary("0011"))`.

What `remove(b)` must do

`Binary` has **no** `operator==`. Two binary strings are equal when they have the same size **and** the same bit at every index. You must compare them yourself, bit by bit. So `011` and `11` are **not** equal (different sizes), even though their values are the same. Two empty strings are equal.

- Walk from the head. Find the **first** node whose string is equal to `b`.
- If there is one: unlink it, delete it, subtract 1 from `size` and return `true`. Later equal strings stay in the list.
- If there is none (this includes an empty list): change nothing and return `false`.
- Deleting the node frees its `Binary` as well. Do not delete anything else.
- Never read a bit outside a string. When the sizes differ, do not compare bits.
- Example: for `0101 -> 0011 -> 101 -> 0011`, `remove("0011")` returns `true` and gives `0101 -> 101 -> 0011`.

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.
- You may not change any header, so write everything in `B_2.cpp`. A small helper function in `B_2.cpp` is allowed. The ban on helpers applies only to the B_1 recursion questions. Make it `static` so that its name stays inside `B_2.cpp`. In the exam, follow the question text.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.

5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example with a segmentation fault), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B2-05. Linked lists: `BinaryStrings::reverse` (100)

For this question, implement the function `reverse` of the class `BinaryStrings` in the file `B_2.cpp`. It reverses the order of the nodes in the list. All the other functions are given in the other files.

UML (only the classes this question needs)

```
BinaryStrings
- head: Node<Binary>*
- size: int
+ BinaryStrings()
+ ~BinaryStrings()
+ prepend(const Binary&): void
+ getHead(): Node<Binary>*&
+ calculatePos(): int
+ getSize(): int
+ reverse(): void   <- implement
+ operator<<(ostream&, const BinaryStrings&): ostream&   {friend}

Node<T>
- data: T
- nextNode: Node<T>*
+ Node(T)
+ ~Node()
+ getData(): T&
+ next(): Node<T>*&

Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `reverse` (the target), `BinaryStrings::getSize` (added so that the tests can print `size`), the `operator<<` functions (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores its own **copy** of the data (`data: T`, not `T*`). `new Node<Binary>(b)` makes a node that holds a copy of `b`. The copy is made by the `Binary` copy constructor.
- `~Node()` is empty. It does **not** delete the next node. When a node is deleted, the `Binary` inside it is destroyed too, and that frees its bool array. So `delete curr;` is all that is needed to free one node and its string.
- `next()` returns a reference to the pointer to the next node (the link), so `curr->next() = ...;` changes the link.
- `getData()` returns a reference to the `Binary` in the node. `curr->getData()[i]` is bit `i` of that string, and `curr->getData().getSize()` is its number of bits.

- `prepend(b)` puts a new node at the front and adds 1 to `size`.
- The destructor `~BinaryStrings()` deletes every node.
- `size` must always equal the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty`, or the strings joined by `->`, for example `000 -> 0011 -> 1101`. It prints no newline. An empty string prints nothing. For example, a list of `0101`, an empty string and `11` prints as follows (two spaces between the arrows):

```
text 0101 -> -> 11
```

- `Binary(string)`: each `'1'` becomes `true`; every other character becomes `false`. `array[0]` is the leftmost bit.
- Inside `BinaryStrings` you can use `head` and `size` directly.
- `operator=` of `Binary`, and the copy constructor and `operator=` of `BinaryStrings`, are private and not defined in this question. Do not use them.

What `reverse()` must do

- After the call, the nodes are in the opposite order. The old last node is the new head, and the old head is the new last node.
- Change the links (`next()` of the nodes and `head`). Do not create new nodes and do not delete nodes. The marking program checks that the same nodes are still used.
- Do not reverse the bits inside a string: `0011` stays `0011`.
- The new last node's `next()` must be `NULL`.
- `size` does not change. An empty list and a list with one node stay the same.
- Example: `000 -> 0011 -> 1101 -> 0101` becomes `0101 -> 1101 -> 0011 -> 000`.

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.
- You may not change any header, so write everything in `B_2.cpp`. A small helper function in `B_2.cpp` is allowed. The ban on helpers applies only to the B_1 recursion questions. Make it `static` so that its name stays inside `B_2.cpp`. In the exam, follow the question text.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example with a segmentation fault), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B2-06. Linked lists: `BinaryStrings::sort` by decimal value (100)

For this question, implement the function `sort` of the class `BinaryStrings` in the file `B_2.cpp`. It sorts the binary strings by their decimal value, smallest first. All the other functions are given in the other files.

UML (only the classes this question needs)

```
BinaryStrings
- head: Node<Binary>*
- size: int
+ BinaryStrings()
+ ~BinaryStrings()
+ prepend(const Binary&): void
+ getHead(): Node<Binary>*&
+ calculatePos(): int
+ getSize(): int
+ sort(): void   <- implement
+ operator<<(ostream&, const BinaryStrings&): ostream&   {friend}

Node<T>
- data: T
- nextNode: Node<T>*
+ Node(T)
+ ~Node()
+ getData(): T&
+ next(): Node<T>*&

Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `sort` (the target), `BinaryStrings::getSize` (added so that the tests can print `size`), the `operator<<` functions (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores its own **copy** of the data (`data: T`, not `T*`). `new Node<Binary>(b)` makes a node that holds a copy of `b`. The copy is made by the `Binary` copy constructor.
- `~Node()` is empty. It does **not** delete the next node. When a node is deleted, the `Binary` inside it is destroyed too, and that frees its bool array. So `delete curr;` is all that is needed to free one node and its string.
- `next()` returns a reference to the pointer to the next node (the link), so `curr->next() = ...;` changes the link.
- `getData()` returns a reference to the `Binary` in the node. `curr->getData()[i]` is bit `i` of that string, and `curr->getData().getSize()` is its number of bits.

- `prepend(b)` puts a new node at the front and adds 1 to `size`.
- The destructor `~BinaryStrings()` deletes every node.
- `size` must always equal the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty`, or the strings joined by `->`, for example `000 -> 0011 -> 1101`. It prints no newline. An empty string prints nothing. For example, a list of `0101`, an empty string and `11` prints as follows (two spaces between the arrows):

```
text 0101 -> -> 11
```

- `Binary(string)`: each `'1'` becomes `true`; every other character becomes `false`. `array[0]` is the leftmost bit.
- Inside `BinaryStrings` you can use `head` and `size` directly.
- `operator=` of `Binary`, and the copy constructor and `operator=` of `BinaryStrings`, are private and not defined in this question. Do not use them.

What `sort()` must do

The decimal value of a string reads the bits as an unsigned binary number, with `array[0]` (the leftmost bit) as the most significant bit. So `0101` is 5, `11` is 3, `011` is 3, `1000` is 8, and `0` and the empty string are 0.

- After the call, the values are in ascending order (smallest at the head).
- Strings with **equal** values keep their original order. Example: `11` comes before `011` if it was before `011` in the list.
- Move whole nodes by changing the links (`next()` of the nodes and `head`). Do not create new nodes and do not delete nodes. The marking program checks that the same nodes are still used.
- Do not swap the data between nodes. `Binary` has no usable `operator=` here, because it is private.
- The last node's `next()` must be `NULL` after the sort.
- `size` does not change. An empty list and a list with one node stay the same.
- The tests use strings of at most 31 bits, so each value fits in an `int`.
- Example: `0101 -> 11 -> 1000 -> 011 -> 0` becomes `0 -> 11 -> 011 -> 0101 -> 1000`.

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.
- You may not change any header, so write everything in `B_2.cpp`. A small helper function in `B_2.cpp` is allowed. The ban on helpers applies only to the B_1 recursion questions. Make it `static` so that its name stays inside `B_2.cpp`. In the exam, follow the question text.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example with a segmentation fault), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B2-07. Linked lists: `BinaryStrings::calculatePos` (100)

For this question, implement the function `calculatePos` of the class `BinaryStrings` in the file `B_2.cpp`. It counts the positive binary strings in the list. All the other functions are given in the other files.

UML (only the classes this question needs)

```
BinaryStrings
- head: Node<Binary>*
- size: int
+ BinaryStrings()
+ ~BinaryStrings()
+ prepend(const Binary&): void
+ getHead(): Node<Binary>*&
+ getSize(): int
+ calculatePos(): int   <- implement
+ operator<<(ostream&, const BinaryStrings&): ostream&   {friend}

Node<T>
- data: T
- nextNode: Node<T>*
+ Node(T)
+ ~Node()
+ getData(): T&
+ next(): Node<T>*&

Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `calculatePos` (the target), `BinaryStrings::getSize` (added so that the tests can print `size`), the `operator<<` functions (not drawn; the exam paper says every class has one), and the `{const}` markers. The exam paper does not say what "Pos" means. In this question it means "positive", as defined below. The real exam's question text decides. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores its own **copy** of the data (`data: T`, not `T*`). `new Node<Binary>(b)` makes a node that holds a copy of `b`. The copy is made by the `Binary` copy constructor.
- `~Node()` is empty. It does **not** delete the next node. When a node is deleted, the `Binary` inside it is destroyed too, and that frees its bool array. So `delete curr;` is all that is needed to free one node and its string.
- `next()` returns a reference to the pointer to the next node (the link), so `curr->next() = ...;` changes the link.

- `getData()` returns a reference to the `Binary` in the node. `curr->getData()[i]` is bit `i` of that string, and `curr->getData().getSize()` is its number of bits.
- `prepend(b)` puts a new node at the front and adds 1 to `size`.
- The destructor `~BinaryStrings()` deletes every node.
- `size` must always equal the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty`, or the strings joined by `->`, for example `000 -> 0011 -> 1101`. It prints no newline. An empty string prints nothing. For example, a list of `0101`, an empty string and `11` prints as follows (two spaces between the arrows):

```
text 0101 -> -> 11
```

- `Binary(string)`: each `'1'` becomes `true`; every other character becomes `false`. `array[0]` is the leftmost bit.
- Inside `BinaryStrings` you can use `head` and `size` directly.
- `operator=` of `Binary`, and the copy constructor and `operator=` of `BinaryStrings`, are private and not defined in this question. Do not use them.

What `calculatePos()` must do

Bit 0 (the leftmost bit) is a sign bit: 1 means negative, 0 means not negative. A binary string is **positive** when bit 0 is `false` (0) **and** at least one other bit is `true` (1).

- So `0011` and `01` are positive. `000` is zero, not positive. `1101` is negative. `0`, `1` and the empty string are not positive.
- Return the number of positive strings in the whole list. An empty list gives 0.
- Count each positive string once, even if it has many 1 bits.
- Never read a bit outside a string. An empty string has no bit 0.
- Do not change the list or any bit.
- Example: for `000 -> 0011 -> 1101 -> 0101`, `calculatePos()` returns 2.

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.
- You may not change any header, so write everything in `B_2.cpp`. A small helper function in `B_2.cpp` is allowed. The ban on helpers applies only to the B_1 recursion questions. Make it `static` so that its name stays inside `B_2.cpp`. In the exam, follow the question text.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example with a

segmentation fault), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B2-08. Linked lists: `BinaryStrings` copy constructor (100)

For this question, implement the copy constructor of the class `BinaryStrings` in the file `B_2.cpp`. It makes a deep copy of a list. All the other functions are given in the other files.

UML (only the classes this question needs)

```
BinaryStrings
- head: Node<Binary>*
- size: int
+ BinaryStrings()
+ BinaryStrings(const BinaryStrings&)   <- implement
+ ~BinaryStrings()
+ prepend(const Binary&): void
+ getHead(): Node<Binary>*&
+ calculatePos(): int
+ getSize(): int
+ operator<<(ostream&, const BinaryStrings&): ostream&   {friend}

Node<T>
- data: T
- nextNode: Node<T>*
+ Node(T)
+ ~Node()
+ getData(): T&
+ next(): Node<T>*&

Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: the copy constructor (the target), `BinaryStrings::getSize` (added so that the tests can print `size`), the `operator<<` functions (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores its own **copy** of the data (`data: T`, not `T*`). `new Node<Binary>(b)` makes a node that holds a copy of `b`. The copy is made by the `Binary` copy constructor.
- `~Node()` is empty. It does **not** delete the next node. When a node is deleted, the `Binary` inside it is destroyed too, and that frees its bool array. So `delete curr;` is all that is needed to free one node and its string.
- `next()` returns a reference to the pointer to the next node (the link), so `curr->next() = ...;` changes the link.

- `getData()` returns a reference to the `Binary` in the node. `curr->getData()[i]` is bit `i` of that string, and `curr->getData().getSize()` is its number of bits.
- `prepend(b)` puts a new node at the front and adds 1 to `size`.
- The destructor `~BinaryStrings()` deletes every node.
- `size` must always equal the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty`, or the strings joined by `->`, for example `000 -> 0011 -> 1101`. It prints no newline. An empty string prints nothing. For example, a list of `0101`, an empty string and `11` prints as follows (two spaces between the arrows):

```
text 0101 -> -> 11
```

- `Binary(string)`: each `'1'` becomes `true`; every other character becomes `false`. `array[0]` is the leftmost bit.
- Inside `BinaryStrings` you can use `head` and `size` directly.
- `operator=` of `Binary` and `operator=` of `BinaryStrings` are private and not defined in this question. Do not use them.

What `BinaryStrings(const BinaryStrings& other)` must do

- Make a **deep copy**: the new list has its own new nodes, one for each node of `other`, holding copies of the same strings in the **same order**.
- The two lists share no node and no bool array. A change to one list (a bit, a new node) must not change the other. Each list's destructor deletes only its own nodes.
- `size` of the new list equals the number of nodes.
- A copy of an empty list is empty: `head` is `NULL` and `size` is 0.
- The last node's `next()` must be `NULL`.
- Remember to set `head` and `size` first. A constructor starts with undefined values in its members.
- `prepend` adds at the front, so copying with `prepend` alone gives the reverse order.
- The copy constructor is also called when a `BinaryStrings` is passed **by value**.

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.
- You may not change any header, so write everything in `B_2.cpp`. A small helper function in `B_2.cpp` is allowed. The ban on helpers applies only to the B_1 recursion questions. Make it `static` so that its name stays inside `B_2.cpp`. In the exam, follow the question text.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example with a segmentation fault), the mark is 0.

B_3 Template specialisation

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B3-01. Template specialisation: `printValue<Binary>` (100)

For this question, write a template specialisation (a version of a function template written for one exact type). The function template `printValue` prints any value with `operator<<`. For a `Binary`, it must also print the decimal value. Write the specialisation `printValue<Binary>` in the file `B_3.cpp`.

UML (only what this question needs)

```
Templates.h (function templates)
+ printValue<T>(const T&): void
+ printValue<Binary>(const Binary&): void   <- implement

Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `printValue` and its specialisation (the target; function templates are not on the UML), `operator<<` (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- The general template in `Templates.h`: `cpp template <class T> void printValue(const T& value) { cout << "Value: " << value << endl; }`
- The declaration of the specialisation, also in `Templates.h`, right after the general template: `cpp template <> void printValue<Binary>(const Binary& value);`
- `operator<<` for a `Binary` prints its bits, for example `0101`, with no newline. `array[0]` is the leftmost bit.
- `operator[]` has a `const` version and `getSize()` is `const`, so both can be used on a `const Binary&`.

What `printValue<Binary>(const Binary& value)` must do

- Print `Binary:`, then the bits, then a space, then the decimal value in parentheses `( )`, then `endl`. Example: for `0101` the line is `Binary: 0101 (5)`.
- The decimal value reads the bits as an unsigned binary number, with `array[0]` (the leftmost bit) as the most significant bit. `11111111` is 255. An empty string has value 0 and prints `Binary: (0)`.
- The tests use at most 31 bits, so the value fits in an `int`.
- Do not change the `Binary`.

How and where to write the specialisation

- Write it in `B_3.cpp`, in the form of the lecture example (Lecture 19): the line `template <>`, then the function with the type in angle brackets after the function name. It must match the declaration in `Templates.h` exactly. With `T = Binary`, the parameter `const T&` becomes `const Binary&`. Your definition starts with the same two lines as the declaration, without the `;`:

`cpp template <> void printValue<Binary>(const Binary& value) {` - Why `B_3.cpp`: a full specialisation is an ordinary function, not a template. It must be defined in exactly one `.cpp` file. Suppose the body were in `Templates.h`. Both `main.cpp` and `B_3.cpp` include `Templates.h`, so each would get its own copy. The link would then fail with `multiple definition`. (An `inline` definition would be allowed; see B3-06.) - Why the declaration is in the header: the specialisation must come after the general template and before the first use (Lecture 19: "placed after the general template function"). Every file that includes `Templates.h` sees the declaration, so `main.cpp` calls your version and not the general one. - Write only the definition. Do not write the general template again, and do not change any header.

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_3` folder.
2. Run `make B_3`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B3-02. Template specialisation: `largest<Binary>` (100)

For this question, write a template specialisation (a version of a function template written for one exact type). The function template `largest` returns the larger of two values with `>`. `Binary` has no `>` operator, so the general template cannot be used for it. Write the specialisation `largest<Binary>` in the file `B_3.cpp`. It compares the two strings by decimal value.

UML (only what this question needs)

```
Templates.h (function templates)
+ largest<T>(T, T): T
+ largest<Binary>(Binary, Binary): Binary   <- implement

Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `largest` and its specialisation (the target; function templates are not on the UML), `operator<<` (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- The general template in `Templates.h`: `cpp template <class T> T largest(T a, T b) { if (b > a) { return b; } return a; }`
- The declaration of the specialisation, also in `Templates.h`, right after the general template: `cpp template <> Binary largest<Binary>(Binary a, Binary b);`
- `Binary` has a deep copy constructor, so passing and returning a `Binary` by value is safe. `operator=` of `Binary` is private and not defined in this question.
- `operator<<` for a `Binary` prints its bits, for example `1000`, with no newline.
- In examples, a call such as `largest("11", "0001")` is short for `largest(Binary("11"), Binary("0001"))`.

What `largest<Binary>(Binary a, Binary b)` must do

- The decimal value reads the bits as an unsigned binary number, with `array[0]` (the leftmost bit) as the most significant bit. `0111` is 7, `1000` is 8, `0001` is 1, and an empty string is 0.
- Return the string with the larger decimal value. Return the original string, not the decimal value: `largest("11", "0001")` returns `11`, not `3`.
- If the two values are equal, return `a`. Example: `largest("011", "11")` returns `011`.
- The number of bits and the number of 1 bits do not decide anything. Only the value does.
- The tests use at most 31 bits, so the value fits in an `int`.
- Print nothing.

How and where to write the specialisation

- Write it in `B_3.cpp`, in the form of the lecture example (Lecture 19): the line `template <>`, then the function with the type in angle brackets after the function name. It must match the declaration in `Templates.h` exactly. The general template takes `T` by value, so the parameters are `Binary a, Binary b`, not references. Your definition starts with the same two lines as the declaration, without the `;`:

`cpp template <> Binary largest<Binary>(Binary a, Binary b) {` - Why `B_3.cpp`: a full specialisation is an ordinary function, not a template. It must be defined in exactly one `.cpp` file. Suppose the body were in `Templates.h`. Both `main.cpp` and `B_3.cpp` include `Templates.h`, so each would get its own copy. The link would then fail with `multiple definition`. (An `inline` definition would be allowed; see B3-06.) - Why the declaration is in the header: the specialisation must come after the general template and before the first use (Lecture 19). Every file that includes `Templates.h` sees the declaration, so `main.cpp` calls your version. - Write only the definition. Do not write the general template again, and do not change any header.

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_3` folder.
2. Run `make B_3`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B3-03. Template specialisation: `isEqual<Binary>` (100)

For this question, write a template specialisation (a version of a function template written for one exact type). The function template `isEqual` compares two values with `==`. `Binary` has no `==` operator, so the general template cannot be used for it. Write the specialisation `isEqual<Binary>` in the file `B_3.cpp`. It compares two strings bit by bit.

UML (only what this question needs)

```
Templates.h (function templates)
+ isEqual<T>(const T&, const T&): bool
+ isEqual<Binary>(const Binary&, const Binary&): bool   <- implement

Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `isEqual` and its specialisation (the target; function templates are not on the UML), `operator<<` (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- The general template in `Templates.h`: `cpp template <class T> bool isEqual(const T& a, const T& b) { return a == b; }`
- The declaration of the specialisation, also in `Templates.h`, right after the general template: `cpp template <> bool isEqual<Binary>(const Binary& a, const Binary& b);`
- `operator[]` has a `const` version and `getSize()` is `const`, so both can be used on a `const Binary&`.
- `Binary(string)`: each `'1'` becomes `true`; every other character becomes `false`.

What `isEqual<Binary>(const Binary& a, const Binary& b)` must do

- Return `true` when `a` and `b` have the same size **and** the same bit at every index. Otherwise return `false`.
- Strings of different sizes are not equal, even with the same value: `011` and `11` are not equal.
- Two empty strings are equal.
- Never read a bit outside a string: when the sizes differ, do not compare bits.
- Do not change either string, and print nothing.

How and where to write the specialisation

- Write it in `B_3.cpp`, in the form of the lecture example (Lecture 19): the line `template <>`, then the function with the type in angle brackets after the function name. It must match the declaration in

`Templates.h` exactly. With `T = Binary`, each `const T&` becomes `const Binary&`. Your definition starts with the same two lines as the declaration, without the `;`:

`cpp template <> bool isEqual<Binary>(const Binary& a, const Binary& b) {` - Do not write `template <class T>` here. That declares a new, different template, and the specialisation that `main.cpp` calls stays undefined (link error). - Why `B_3.cpp`: a full specialisation is an ordinary function, not a template. It must be defined in exactly one `.cpp` file. Suppose the body were in `Templates.h`. Both `main.cpp` and `B_3.cpp` include `Templates.h`, so each would get its own copy. The link would then fail with `multiple definition`. (An `inline` definition would be allowed; see B3-06.) - Why the declaration is in the header: the specialisation must come after the general template and before the first use (Lecture 19). Every file that includes `Templates.h` sees the declaration, so `main.cpp` calls your version. - Write only the definition. Do not write the general template again, and do not change any header.

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_3` folder.
2. Run `make B_3`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B3-04. Template specialisation: `addition<Binary, Binary>` throws (100)

For this question, write a template specialisation (a version of a function template written for exact types). The lecture example does the same for two `char` values: it throws an exception. Adding two binary strings is not allowed in this system. Write the specialisation of `addition` for two `Binary` objects in the file `B_3.cpp`. It must throw.

UML (only what this question needs)

```
Templates.h (function templates)
+ addition<T, U>(T, U): T
+ addition<Binary, Binary>(Binary, Binary): Binary   <- implement

Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `addition` and its specialisation (the target; function templates are not on the UML), `operator<<` (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- The general template in `Templates.h` (from the lectures): `cpp template <class T, class U> T addition(T v1, U v2) { return v1 + v2; }`
- The declaration of the specialisation, also in `Templates.h`, right after the general template: `cpp template <> Binary addition<Binary, Binary>(Binary v1, Binary v2);`
- `Binary` has no `+` operator. The section B UML has no exception class.

What `addition<Binary, Binary>(Binary v1, Binary v2)` must do

- Do not add anything. Throw the C-string `"Cannot add two binary strings"`, as the lecture example throws `"Cannot add two chars"`.
- `main.cpp` catches it with `catch (const char* msg)`. A C-string literal is thrown as a `const char*`, so this catch matches it.
- Do not throw a `std::string`, a `Binary` or a pointer made with `new`. None of these match `catch (const char* msg)`.
- Do not print anything.

How and where to write the specialisation

- Write it in `B_3.cpp`, in the form of the lecture example (Lecture 19): the line `template <>`, then the function with **both** types in angle brackets after the function name. It must match the declaration in `Templates.h` exactly (return type, name, `<Binary, Binary>` and parameter types). Your definition starts with the same two lines as the declaration, without the `;`:

`cpp template <> Binary addition<Binary, Binary>(Binary v1, Binary v2) {` - The return type `Binary` is still required, even though the function only throws. - Why `B_3.cpp`: a full specialisation is an ordinary function, not a template. It must be defined in exactly one `.cpp` file. Suppose the body were in `Templates.h`. Both `main.cpp` and `B_3.cpp` include `Templates.h`, so each would get its own copy. The link would then fail with `multiple definition`. (An `inline` definition would be allowed; see B3-06.) - Why the declaration is in the header: Lecture 19 says specialisations must be "placed after the general template function". They must also come before the first use. Every file that includes `Templates.h` sees the declaration, so `main.cpp` calls your version. - Write only the definition. Do not write the general template again, and do not change any header.

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_3` folder.
2. Run `make B_3`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B3-05. Template specialisation: member `Node<Binary>::toInt` (100)

For this question, write a specialisation of **one member function** of a class template (a version of that member written for one exact type). `Node<T>::toInt()` converts the data of a node to an `int`. For `Node<Binary>`, the general version cannot convert a `Binary`. Write the specialisation `Node<Binary>::toInt()` in the file `B_3.cpp`. It returns the decimal value of the stored string.

UML (only the classes this question needs)

```
Node<T>
- data: T
- nextNode: Node<T>*
+ Node(T)
+ ~Node()
+ getData(): T&
+ next(): Node<T>*&
+ toInt(): int

Node<Binary>   (specialisation of one member)
+ toInt(): int   <- implement

BinaryStrings
- head: Node<Binary>*
- size: int
+ BinaryStrings()
+ ~BinaryStrings()
+ prepend(const Binary&): void
+ getHead(): Node<Binary>*&
+ calculatePos(): int
+ operator<<(ostream&, const BinaryStrings&): ostream&   {friend}

Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `Node<T>::toInt` and its specialisation for `Binary` (the target), the `operator<<` functions (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- The general `toInt()` in `Node.cpp`: `cpp template <class T> int Node<T>::toInt() { return (int)data; }` For `Node<int>` it returns the value, for `Node<double>` it drops the fraction (3.9 gives 3), for `Node<bool>` it gives 1 or 0, and for `Node<char>` the character code.
- The declaration of the specialisation at the bottom of `Node.h`, after `#include "Node.cpp"`: `cpp template <> int Node<Binary>::toInt();` `Node.h` includes `Binary.h` at the top, because the

declaration names `Node<Binary>`, and a `Node<Binary>` holds a whole `Binary`.
- Inside a member of `Node<Binary>`, `data` is the stored `Binary`. `data[i]` is bit `i`, and `data.getSize()` is the number of bits.

What `Node<Binary>::toInt()` must do

- Return the decimal value of `data`, read as an unsigned binary number with `data[0]` (the leftmost bit) as the most significant bit. `1010` gives 10, `0011` gives 3, and an empty string gives 0.
- The tests use at most 31 bits, so the value fits in an `int`.
- Do not change `data`, and print nothing.

How and where to write the specialisation

- Write it in `B_3.cpp`.
- The lecture rule (Lecture 19) is `template <>` followed by the concrete type. The slides show only a function (`addition<char, char>`). For a member of a class template, the type goes after the class name.
- Write the line `template <>`, then the return type, then the class name **with the type** `Node<Binary>`, then `::toInt()`. The type goes after the **class** name, not after the function name.
- Your definition starts with the same two lines as the declaration in `Node.h`, without the `;`:

`cpp template <> int Node<Binary>::toInt() {` - Do not write `template <class T>`. Do not write `Node::toInt<Binary>()`. Both give compile errors. - Why `B_3.cpp`: the specialisation is an ordinary member function, not a template. It must be defined in exactly one `.cpp` file. Suppose the body were written at the end of `Node.cpp`. The link then fails with `undefined reference to 'Node<Binary>::toInt()'`. Suppose instead the body were written in `Node.h`. Every file that includes `Node.h` gets its own copy, and the link fails with `multiple definition of 'Node<Binary>::toInt()'`. (An `inline` definition would be allowed; see B3-06.) You upload only `B_3.cpp`, so the definition goes there. - Why the declaration is in `Node.h`: the specialisation must come after the general template and before the first use. Every file that includes `Node.h` sees it, so `main.cpp` calls your version. - Write only the definition. Do not change any header.

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_3` folder.
2. Run `make B_3`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of

an uncaught exception), the mark is 0.

Practice question based on the published re-practical scenario. The real exam's complete UML may differ.

B3-06. Template specialisation: `countTrue<Binary>` in a file that a header includes (100)

For this question, write a template specialisation (a version of a function template written for one exact type). The function template `countTrue` walks a list of nodes and counts the nodes whose data is true. For `Binary`, it must count the true **bits** in all the strings instead. Write the specialisation `countTrue<Binary>` in the file `B_3.cpp`.

In this question, `Templates.h` ends with `#include "B_3.cpp"`. Template class headers often do the same with their `.cpp` file. Read "How and where to write the specialisation": the definition must be `inline`.

UML (only what this question needs)

```
Templates.h (function templates)
+ countTrue<T>(Node<T>*): int
+ countTrue<Binary>(Node<Binary>*): int   <- implement

Node<T>
- data: T
- nextNode: Node<T>*
+ Node(T)
+ ~Node()
+ getData(): T&
+ next(): Node<T>*&

BinaryStrings
- head: Node<Binary>*
- size: int
+ BinaryStrings()
+ ~BinaryStrings()
+ prepend(const Binary&): void
+ getHead(): Node<Binary>*&
+ calculatePos(): int
+ operator<<(ostream&, const BinaryStrings&): ostream&   {friend}

Binary
- array: bool*
- size: int
+ Binary(string)
+ Binary(const Binary&)
+ ~Binary()
+ operator[](int): bool&
+ operator[](int): const bool&   {const}
+ getSize(): int   {const}
+ operator<<(ostream&, const Binary&): ostream&   {friend}
```

Not on the partial UML in the exam paper: `countTrue` and its specialisation (the target; function templates are not on the UML), the `operator<<` functions (not drawn; the exam paper says every class has one), and the `{const}` markers. The real exam's complete UML may add members like these.

What is given

- The general template in `Templates.h`: `cpp template <class T> int countTrue(Node<T>* head) { int count = 0; Node<T>* curr = head; while (curr != NULL) { if (curr->getData()) { count++; } curr = curr->next(); } return count; }` For `Node<int>` it counts the values that are not 0. A `Binary` cannot be used in an `if`, so the general template does not compile for `Binary`.
- After it, also in `Templates.h`, the declaration of the specialisation and then the include of your file: ```cpp template <> int countTrue(Node* head);

#include "B_3.cpp" `` - The starter B_3.cpp has an include guard (#ifndef B_3_CPP, #define B_3_CPP, #endif) and #include "Templates.h". B_3.cpp is also compiled on its own (it is on the g++ line in the makefile). The include guard stops the file from being read twice in one compilation. - getHead() returns the first node of a BinaryStrings list. curr->getData() is the Binary in a node, curr->getData()[i] is bit i, and curr->getData().getSize()` is its number of bits.

What `countTrue<Binary>(Node<Binary>* head)` must do

- Walk from `head` to the end of the list (`next()` is `NULL` after the last node).
- Return the total number of `true` bits in all the strings. Example: `000 -> 0011 -> 1101 -> 0101` gives 0 + 2 + 3 + 2 = 7.
- `NULL` gives 0. Empty strings add 0.
- Do not change the list, and print nothing.

How and where to write the specialisation

- Write it in `B_3.cpp`, **between** `#define B_3_CPP` and `#endif`. Code after `#endif` is read twice when `B_3.cpp` is compiled on its own, and gives a redefinition error.
- Use the Lecture 19 form (`template <>`, then the function with the type in angle brackets after the function name), and add the keyword `inline` before the return type:

`cpp template <> inline int countTrue<Binary>(Node<Binary>* head) {` - Why `inline`: a full specialisation is an ordinary function, not a template. Here its definition is in `B_3.cpp`, and `Templates.h` includes `B_3.cpp`. So every `.cpp` file that includes `Templates.h` (here `main.cpp` and `B_3.cpp` itself) gets its own copy of the definition. Without `inline`, the link fails with `multiple definition of 'int countTrue<Binary>(Node<Binary>*)'`. `inline` allows one identical definition in each file. - The declaration in `Templates.h` comes after the general template and before any use, as Lecture 19 requires. Your definition comes right after it, through the include. - Do not write the general template again, do not remove the include guard, and do not change any header.

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.

The `operator<<` functions and the `ExpectedOutput` block apply to this question only. Other questions may print the same class differently.

Build and upload

1. Open a terminal in the `B_3` folder.
2. Run `make B_3`.

3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.